

AWS en pratique

IAM, la CLI et les rôles

IAM, les fondamentaux. La CLI et les rôles.

Identités

utilisateur, groupe, rôle

Politiques

lire et écrire le JSON

Évaluation

refus, moindre privilège

CLI

connexion, profils, filtres

Rôles

STS, rôles de service

Support de formation, version 1.3, octobre 2026. Parcours « AWS en pratique ». Fait suite au support « Cloud et console ».

IAM, les fondamentaux

- Distinguer utilisateur, groupe, rôle et politique, et dire ce qui se rattache à quoi.
- Lire une politique JSON élément par élément : Effect, Action, Resource, Condition.
- Prédire le résultat d'une requête à partir des politiques en présence.
- Expliquer le refus par défaut, le refus explicite et le moindre privilège.
- Choisir entre politique gérée par AWS, gérée par le client et en ligne.

Prérequis. Les chapitres « Comprendre le cloud » et « Premiers pas dans la console » : compte AWS, utilisateur racine, console, région, ARN, CloudShell, étiquettes.

La CLI et les rôles

- Installer la CLI et s'y connecter sans clé d'accès permanente.
- Utiliser les profils, régler la région et le format de sortie.
- Extraire l'information utile avec `--query` et `--output`.
- Expliquer comment STS délivre des identifiants temporaires à qui endosse un rôle.
- Décrire comment EC2, Lambda et ECS obtiennent leurs droits par un rôle de service.

Ce support ne contient que la théorie. Les travaux pratiques 3 et 4 font l'objet d'énoncés séparés. Les commandes montrées servent d'illustration.

IAM, les fondamentaux

IAM, vue d'ensemble	Authentifier, autoriser, vocabulaire, objets.
Les identités	Utilisateur, groupe, rôle ; identifiants.
Lire une politique	Structure JSON, Effect, Action, Resource, Condition.
Évaluation des droits	Refus par défaut, refus explicite, garde-fous, moindre privilège.
Types de politiques	Gérées par AWS, gérées par le client, en ligne.

La CLI et les rôles

La CLI	Installation, <code>aws login</code> , profils, région, format.
Filtrer les résultats	<code>--filters</code> , <code>--query</code> (JMESPath), <code>--output</code> .
Rôles et STS	Trust policy, AssumeRole, identifiants temporaires.
Rôles de service	EC2, Lambda, ECS ; <code>iam:PassRole</code> .

Contexte de la formation. Chaque apprenant dispose d'un utilisateur IAM (`user1` à `user15`) et prend le rôle `RoleApprenants` . Ces deux chapitres expliquent ce qui se passe réellement derrière ces deux gestes.

Navigation. Flèches gauche et droite ou barre d'espace, **M** pour le sommaire, **F** pour le plein écran.

IAM, les fondamentaux, partie 1

IAM, vue d'ensemble

Le service qui répond à deux questions pour chaque requête : qui êtes-vous, et en avez-vous le droit ?

- Authentification et autorisation
- Les propriétés du service
- Vocabulaire : principal, action, ressource
- Les objets d'IAM et leurs liens
- Le contexte d'une requête
- Ce qu'IAM ne fait pas

Qu'est-ce qu'IAM

Définition : IAM (Identity and Access Management)

Service d'AWS qui gère les identités (qui peut se présenter) et les autorisations (ce que chacune peut faire, sur quelles ressources et dans quelles conditions).

Définition : authentification

Vérification de l'identité de celui qui envoie une requête : mot de passe et second facteur dans la console, signature de la requête par une clé dans la CLI et les SDK.

Définition : autorisation

Décision d'accepter ou de refuser une action demandée par une identité authentifiée, d'après les politiques qui s'appliquent.

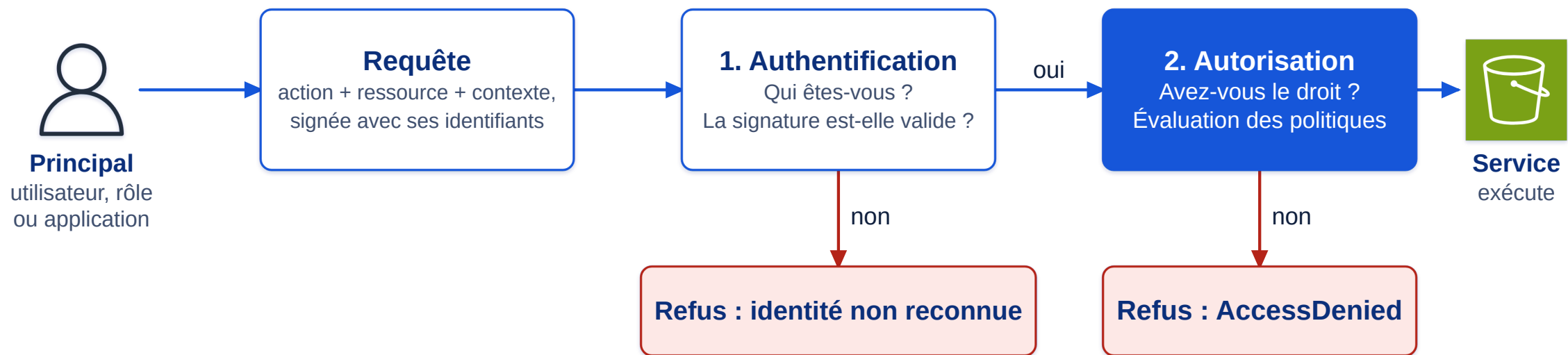
Pourquoi IAM est central

- Toute action sur AWS est un appel d'API, et tout appel d'API passe par IAM.
- Aucun service n'a son propre système de droits : EC2, S3 ou RDS s'en remettent tous à IAM.
- Dans le modèle de responsabilité partagée, IAM est entièrement du côté du client.

Ce que l'on y fait

- Créer des identités pour les personnes et pour les applications.
- Écrire des politiques qui décrivent les droits.
- Attacher les politiques aux identités ou aux ressources.
- Vérifier, auditer et réduire les droits accordés.

Schéma : authentifier, puis autoriser



La requête

Elle dit ce que l'on veut faire (l'action), sur quoi (la ressource), et transporte un contexte : heure, adresse IP, région, présence d'un second facteur.

Étape 1

AWS vérifie la signature de la requête. Si les identifiants sont inconnus, expirés ou mal signés, la requête est rejetée avant tout examen des droits.

Étape 2

AWS rassemble toutes les politiques concernées et les évalue. Sans autorisation explicite, la réponse est non : c'est le refus par défaut.

Propriété	Conséquence pratique
Service mondial	Un utilisateur, un rôle ou une politique existe pour tout le compte, quelle que soit la région. La console affiche « Global ».
Gratuit	Créer des identités et des politiques ne coûte rien. On peut donc découper finement.
Refus par défaut	Une identité neuve ne peut rien faire tant qu'on ne lui a rien accordé.
Cohérence à terme	Une modification met quelques secondes à se propager partout. Un rôle créé à l'instant peut être refusé pendant un court moment.
Granularité fine	On peut autoriser une seule action, sur une seule ressource, à une condition précise.
Intégré à tous les services	Un seul langage de politique pour l'ensemble d'AWS.

Définition : cohérence à terme

Propriété d'un système réparti dans lequel une modification n'est pas visible partout instantanément, mais le devient après un court délai.

Conséquence dans les scripts. Juste après avoir créé un rôle, une commande qui l'utilise peut échouer. On attend quelques secondes ou on réessaie : ce n'est pas une erreur de droits.

Rappel. L'utilisateur racine échappe à IAM : aucune politique ne le limite. Tout ce qui suit concerne les identités créées dans IAM.

Vocabulaire : principal, action, ressource

Définition : principal

Entité qui envoie une requête à AWS et que l'on peut authentifier : utilisateur racine, utilisateur IAM, session d'un rôle, service AWS, identité fédérée.

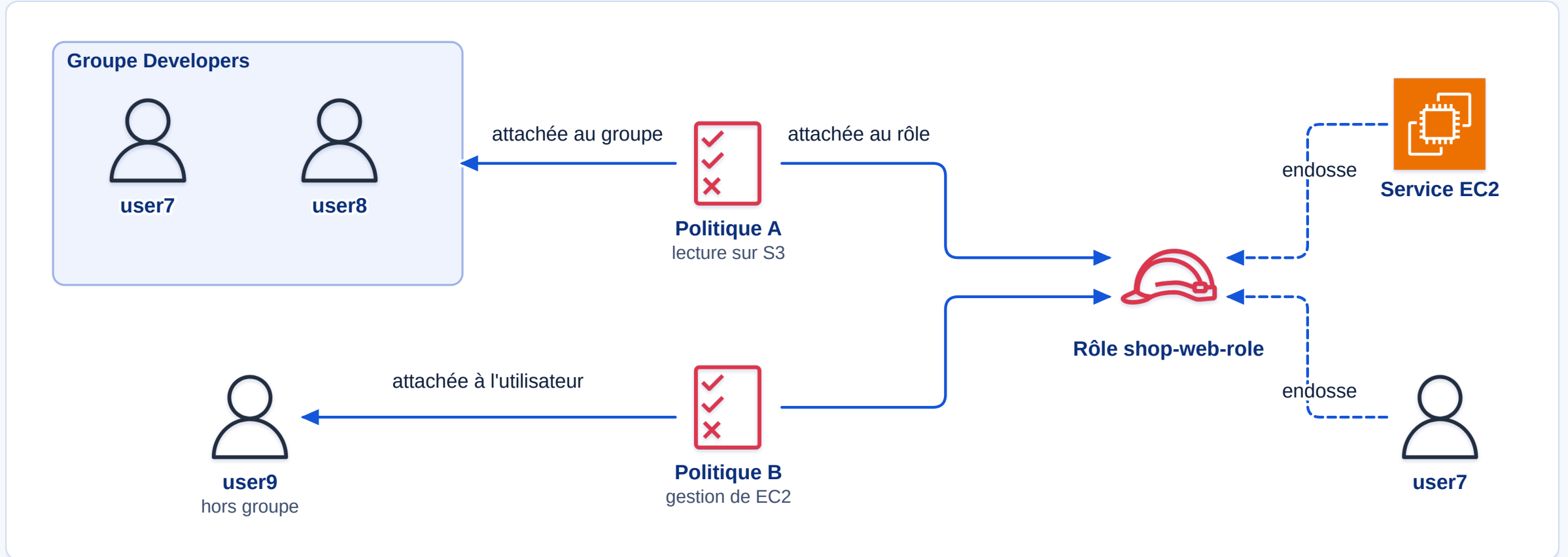
Définition : identité IAM

Objet d'IAM auquel on attache des politiques : un utilisateur, un groupe ou un rôle.

Terme	Définition	Exemple
Action	Opération d'un service, nommée <code>service:Opération</code> . C'est ce que la politique autorise ou refuse.	<code>s3:GetObject</code> , <code>ec2:RunInstances</code>
Ressource	Objet sur lequel porte l'action, désigné par son ARN.	<code>arn:aws:s3:::shop-images/logo.png</code>
Requête	Un appel d'API : principal, action, ressource et données de contexte.	user7 demande <code>s3:GetObject</code> sur le logo
Politique	Document JSON qui décrit des autorisations ou des refus.	<code>AmazonS3ReadOnlyAccess</code>
Session	Période pendant laquelle des identifiants temporaires sont valables.	Une heure après la prise d'un rôle
Entité	Identité qui peut s'authentifier : utilisateur ou rôle (pas le groupe).	<code>user7</code> , <code>RoleApprenants</code>

Nuance utile. Un groupe est une identité (on lui attache des politiques) mais pas un principal : on ne peut pas « se connecter en tant que groupe », ni désigner un groupe dans une politique de ressource.

Schéma : les objets d'IAM et leurs liens



Les politiques portent les droits

Une même politique peut être attachée à plusieurs groupes, utilisateurs et rôles. La modifier change les droits de tous ses porteurs.

Les groupes rassemblent

Un utilisateur hérite des politiques de tous ses groupes, en plus de celles qui lui sont attachées directement.

Les rôles se prêtent

Un rôle n'appartient à personne : il est endossé un temps par un utilisateur, un service ou une identité externe.

Ce qu'AWS examine dans une requête

Élément	Exemple pour une lecture dans S3
Principal	arn:aws:iam::123456789012:user/user7
Action	s3:GetObject
Ressource	arn:aws:s3:::shop-images/logo.png
Données du principal	Ses étiquettes, ses groupes, les politiques attachées.
Données de la ressource	Ses étiquettes, sa politique de ressource si elle en a une.
Données d'environnement	Adresse IP source, date et heure, région visée, chiffrement de la connexion, second facteur présenté.

Définition : contexte de la requête

Ensemble des informations qu'AWS réunit sur un appel d'API pour le comparer aux politiques : qui, quoi, sur quoi, et dans quelles circonstances.

À quoi cela sert

- Les éléments Action et Resource d'une politique sont comparés à l'action et à la ressource de la requête.
- L'élément Condition est comparé aux données de contexte.
- On peut ainsi écrire : « autoriser seulement depuis le réseau de l'entreprise », « seulement avec un second facteur », « seulement dans les régions d'Europe ».

Ce qu'IAM ne fait pas

Besoin	Ce n'est pas IAM, c'est...	Pourquoi
Inscrire et connecter les clients de mon site web	Amazon Cognito	IAM sert à administrer AWS, pas à gérer des millions d'utilisateurs finaux.
Donner à mes 300 salariés un accès unique à 20 comptes AWS	IAM Identity Center	Il s'appuie sur l'annuaire de l'entreprise et crée lui-même les rôles nécessaires.
Gérer les comptes Linux ou Windows d'une instance	Le système de l'instance, ou Systems Manager	IAM contrôle l'API d'AWS, pas ce qui se passe à l'intérieur d'une machine.
Gérer les utilisateurs SQL d'une base de données	Le moteur de la base	Même raison, sauf authentification IAM activée sur la base.
Filtrer le trafic réseau	Security groups, network ACL	IAM autorise des appels d'API, pas des paquets.
Limiter ce que peut faire un compte entier	AWS Organizations (SCP)	Les SCP s'appliquent au-dessus d'IAM, utilisateur racine compris.

Deux plans à distinguer. IAM protège le **plan de contrôle** : créer, modifier, supprimer des ressources. L'accès au **plan de données** (se connecter en SSH, lire une table SQL) relève souvent d'un autre mécanisme. S3, DynamoDB ou SQS font exception : leurs données aussi se lisent par l'API, donc sous le contrôle d'IAM.

IAM, les fondamentaux, partie 2

Les identités : utilisateur, groupe, rôle

Trois objets pour représenter qui agit, et un quatrième, la politique, pour dire ce qu'il peut faire.

- L'utilisateur IAM et ses identifiants
- Clés d'accès et identifiants temporaires
- Le groupe
- Le rôle
- Utilisateur ou rôle
- La politique et ses types

Définition : utilisateur IAM

Identité créée dans un compte pour représenter une personne ou une application, dotée d'identifiants permanents qui lui sont propres.

Attribut	Exemple
Nom	<code>user7</code> , unique dans le compte, 64 caractères au plus
ARN	<code>arn:aws:iam::123456789012:user/user7</code>
Identifiant unique	<code>AIDA...</code> , attribué par AWS, jamais réutilisé
Chemin (facultatif)	<code>/formation/</code> , pour classer
Étiquettes	<code>Team = dev</code>

Un utilisateur neuf ne peut rien faire. Il n'a ni mot de passe, ni clé, ni droit : tout est à accorder explicitement.

Règles d'usage

- **Un utilisateur par personne** : jamais de compte partagé, sinon les journaux ne disent plus qui a fait quoi.
- Les droits se donnent par les **groupes**, pas utilisateur par utilisateur.
- Pour les personnes, préférer en entreprise IAM Identity Center ; pour les applications qui tournent sur AWS, un **rôle**.
- L'utilisateur IAM reste adapté aux petits comptes, aux formations et à quelques outils externes qui ne savent pas faire autrement.

Quota. 5 000 utilisateurs par compte : un signe de plus qu'IAM n'est pas fait pour des utilisateurs finaux.

Les identifiants d'un utilisateur IAM

Identifiant	Sert à	Nombre	Remarques
Mot de passe	Se connecter à la console.	1	Soumis à la politique de mot de passe du compte. Facultatif : un utilisateur d'application n'en a pas.
Dispositif MFA	Second facteur de la console, exigible aussi pour certaines actions.	Jusqu'à 8	Passkey, clé de sécurité, application de codes.
Clé d'accès	Signer les appels de la CLI, des SDK et des outils.	2 au plus	Deux, pour pouvoir en changer sans coupure. Le secret n'est affiché qu'à la création.
Clé SSH, identifiants Git	Accéder à des dépôts de code hébergés par AWS.	Quelques-unes	Usage marginal.
Certificat X.509	Anciens protocoles de certains services.	Rare	À ignorer en pratique.

Définition : clé d'accès (access key)

Couple formé d'un identifiant public (`AKIA...`) et d'un secret, qui permet à un programme de signer ses requêtes au nom d'un utilisateur IAM, sans limite de durée.

Le rapport d'identifiants. IAM produit un fichier qui liste, pour chaque utilisateur, l'âge du mot de passe et des clés, leur dernière utilisation et la présence d'un MFA. C'est le point de départ de tout audit.

Schéma : clé à longue durée ou identifiants temporaires

Clé d'accès à longue durée (utilisateur IAM)



AccessKeyId	AKIAIOSFODNN7EXAMPLE
SecretAccessKey	wJalrXUtnFEMI/K7MDENG...

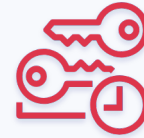
Valable jusqu'à sa suppression : des mois, des années.

Stockée dans un fichier du poste, parfois dans le code.

En cas de fuite : utilisable tant que personne ne la révoque.

Préfixe AKIA

Identifiants temporaires (rôle, console, aws login)



AccessKeyId	ASIAIOSFODNN7EXAMPLE
SecretAccessKey	wJalrXUtnFEMI/K7MDENG...
SessionToken	IQoJb3JpZ2luX2VjEJr...
Expiration	2026-10-05T18:00:00Z

Expirent seuls : de 15 minutes à 12 heures.

Renouvelés automatiquement par la CLI et les SDK.

En cas de fuite : inutilisables peu après.

Préfixe ASIA, avec un jeton de session

La règle actuelle d'AWS. Éviter les clés à longue durée partout où une autre solution existe : `aws login` ou Identity Center pour les personnes, rôles pour les applications. Une clé oubliée dans un dépôt Git est repérée et exploitée par des robots en quelques minutes.

Si une clé est malgré tout nécessaire. Une par usage, des droits minimaux, une rotation régulière, une alerte sur son utilisation, et suppression dès qu'elle ne sert plus. La date de dernière utilisation est visible dans la console.

Définition : groupe IAM

Ensemble d'utilisateurs IAM auquel on attache des politiques. Chaque membre reçoit les droits du groupe tant qu'il en fait partie.

Ce qu'un groupe permet

- Donner les mêmes droits à plusieurs personnes en une seule opération.
- Changer les droits d'une équipe en modifiant un seul objet.
- Gérer une arrivée ou un départ en ajoutant ou retirant un membre.
- Rendre les droits lisibles : le nom du groupe dit la fonction.

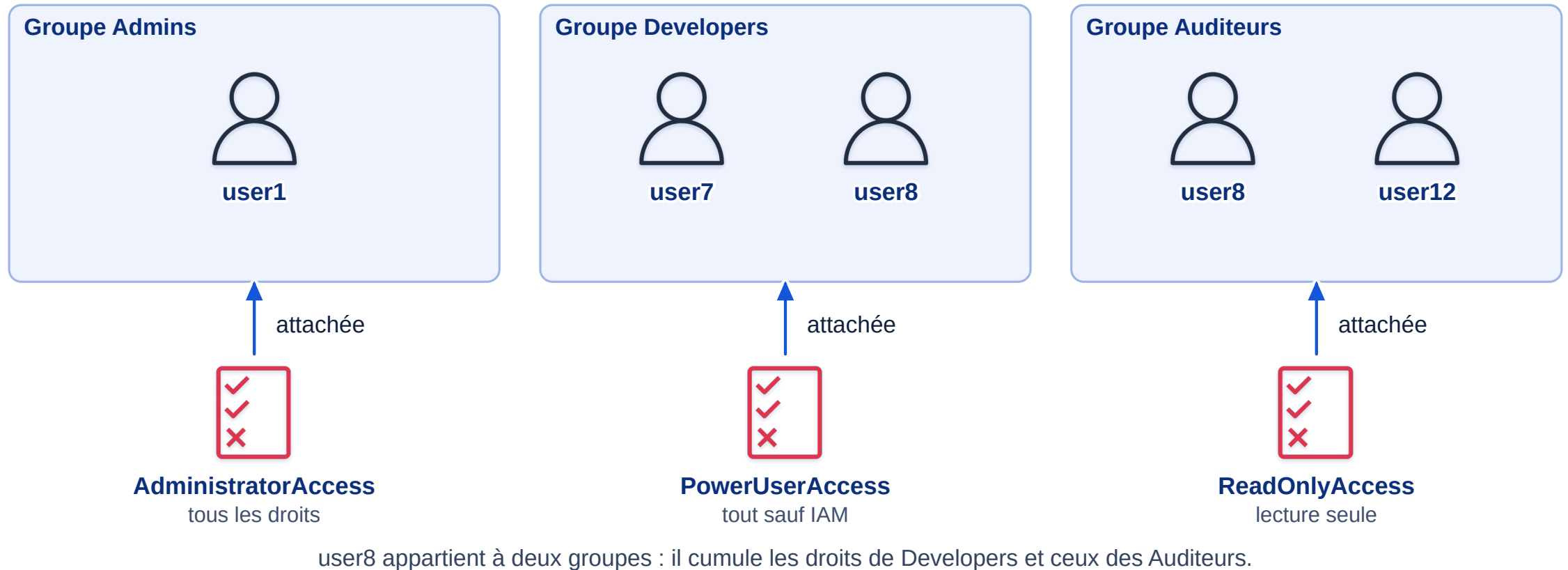
Ce qu'un groupe n'est pas

- **Pas un principal** : on ne s'authentifie pas avec un groupe, et on ne peut pas le citer dans une politique de ressource ni dans une trust policy.
- **Pas imbriquable** : un groupe ne contient que des utilisateurs, pas d'autres groupes.
- **Pas pour les rôles** : un rôle ne peut pas être membre d'un groupe.

Quota	Valeur par défaut
Groupes par compte	300
Groupes par utilisateur	10
Politiques gérées attachées à un groupe	10

Règle de base. Aucune politique attachée directement à un utilisateur : tout passe par des groupes nommés d'après une fonction. Les exceptions se documentent.

Schéma : les groupes portent les droits



Lecture. Les politiques sont attachées aux groupes, jamais aux personnes. Pour savoir ce que peut faire user8, on additionne les politiques de tous ses groupes.

Organisation type. Un groupe par fonction : administrateurs, développeurs, exploitation, auditeurs, facturation. Les groupes par projet s'y ajoutent quand les ressources doivent être cloisonnées.

Définition : rôle IAM

Identité dotée de droits mais dépourvue d'identifiants permanents, destinée à être endossée temporairement par un principal de confiance.

Ce qui le distingue d'un utilisateur

- Ni mot de passe, ni clé d'accès : on ne « se connecte » pas avec un rôle.
- Il n'est lié à personne : plusieurs principaux peuvent l'endosser en même temps.
- Celui qui l'endosse reçoit des **identifiants temporaires**, délivrés par le service STS.
- Pendant la session, seuls les droits du rôle s'appliquent : ceux de l'identité d'origine sont mis de côté.

Deux documents le définissent

- La **trust policy** dit **qui** a le droit d'endosser le rôle.
- Les **politiques de permissions** disent **ce que** le rôle peut faire.

Définition : trust policy (politique d'approbation)

Politique de ressource attachée à un rôle, qui liste les principaux autorisés à l'endosser.

Image. Un rôle est un badge d'accès accroché au mur : la trust policy dit qui a le droit de le décrocher, les permissions disent quelles portes il ouvre, et il faut le rendre à la fin de la session.

Schéma : anatomie d'un rôle



`{ }` Trust policy : le service EC2 peut endosser ce rôle

```
{ "Version": "2012-10-17",
  "Statement": [ {
    "Effect": "Allow",
    "Principal": { "Service": "ec2.amazonaws.com" },
    "Action": "sts:AssumeRole"
  } ] }
```

`{ }` Politique de permissions : lire un bucket

```
{ "Version": "2012-10-17",
  "Statement": [ {
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::shop-images/*"
  } ] }
```

Qui peut endosser un rôle



Un utilisateur ou un rôle du même compte
Un développeur prend temporairement un rôle d'administration pour une opération sensible. C'est le cas de `RoleApprenants`.



Un principal d'un autre compte AWS
Une équipe d'audit, dont les utilisateurs sont dans un compte central, intervient dans les comptes de production.



Un service AWS
Une instance EC2, une fonction Lambda ou une tâche ECS agissent avec les droits d'un rôle : c'est le rôle de service.



Une identité fédérée
Un salarié authentifié par l'annuaire de l'entreprise (SAML), ou une chaîne d'intégration continue authentifiée par OIDC.

Qui endosse	Champ Principal de la trust policy
Un utilisateur précis	<code>{ "AWS": "arn:aws:iam::123456789012:user/user7" }</code>
Tout un compte	<code>{ "AWS": "arn:aws:iam::123456789012:root" }</code> : le compte décide ensuite lesquelles de ses identités y ont droit
Un service	<code>{ "Service": "lambda.amazonaws.com" }</code>
Une identité fédérée	<code>{ "Federated": "arn:aws:iam::123456789012:saml-provider/Entreprise" }</code>

	Utilisateur IAM	Rôle IAM
Représente	Une personne ou une application précise.	Une fonction, un ensemble de droits à prêter.
Identifiants	Permanents : mot de passe, clés d'accès.	Aucun en propre ; temporaires à chaque session.
Durée de validité	Jusqu'à révocation.	De 15 minutes à 12 heures, puis expiration automatique.
Utilisable par	Son seul titulaire.	Tous les principaux admis par la trust policy.
Membre de groupes	Oui.	Non.
Risque en cas de fuite	Élevé : la clé reste valable.	Limité dans le temps.
Traçabilité	Le nom de l'utilisateur.	Le nom du rôle et le nom de la session.
Usage recommandé	Cas restreints.	Le cas général : services, accès entre comptes, fédération, élévation ponctuelle de droits.

Ligne directrice d’AWS. Les personnes passent par une fédération, les charges de travail par des rôles. Dans les deux cas, uniquement des identifiants temporaires.

Dans la formation. `user7` est un utilisateur IAM presque sans droits ; il prend `RoLeApprenants` pour travailler. On retrouve le schéma : une identité stable, des droits prêtés.

Définition : politique (policy)
Document JSON qui énumère des autorisations ou des refus. Attachée à une identité ou à une ressource, elle est évaluée à chaque requête.

Type	Attachée à	Rôle	Accorde des droits ?
Basée sur l'identité	Utilisateur, groupe, rôle	Dit ce que l'identité peut faire.	Oui
Basée sur la ressource	Bucket S3, file SQS, clé KMS, rôle (trust policy)...	Dit qui peut agir sur la ressource.	Oui
Permissions boundary	Utilisateur ou rôle	Plafonne les droits de cette identité.	Non : limite seulement
SCP, RCP (Organizations)	Compte ou unité d'organisation	Plafonnent les droits de tout un compte.	Non : limite seulement
Politique de session	Passée au moment d'endosser un rôle	Restreint la session en cours.	Non : limite seulement

Deux familles. Les deux premières **donnent** des droits. Les trois suivantes sont des **garde-fous** : elles ne donnent rien, elles fixent une limite que les autres ne peuvent pas dépasser.

Dans ce chapitre, l'essentiel porte sur les politiques basées sur l'identité. Les autres sont situées pour comprendre l'évaluation.

Schéma : politique d'identité et politique de ressource



Même compte : il suffit que l'une des deux autorise.

Comptes différents : il faut que les deux autorisent.

Politique basée sur l'identité

Répond à « que peut faire cette identité ? ». Elle voyage avec l'utilisateur, le groupe ou le rôle. C'est le cas le plus fréquent, et le seul possible pour les services sans politique de ressource, comme EC2.

Politique basée sur la ressource

Répond à « qui peut agir sur cette ressource ? ». Elle est écrite sur la ressource elle-même, et c'est le moyen d'ouvrir un accès à un autre compte. Disponible pour S3, SQS, SNS, Lambda, KMS, Secrets Manager et quelques autres.

Identités

Les quatre objets d'IAM en un tableau

Objet	Répond à la question	Contient	Exemple dans la formation
Utilisateur	Qui est cette personne ou cette application ?	Des identifiants permanents, des étiquettes.	user7
Groupe	Quelle est sa fonction ?	Des utilisateurs.	Apprenants
Rôle	Quels droits peut-on emprunter, et qui le peut ?	Une trust policy.	RoleApprenants
Politique	Qu'est-il permis ou interdit de faire ?	Des déclarations : effet, action, ressource, condition.	PowerUserAccess

Qui porte des politiques ?

Les trois identités : utilisateur, groupe et rôle.
Et certaines ressources, par leur politique de ressource.

Qui peut s'authentifier ?

L'utilisateur (identifiants permanents) et la session d'un rôle (identifiants temporaires). Pas le groupe.

Qui apparaît dans les journaux ?

Le principal réel : user/user7 , ou assumed-role/RoleApprenants/user7 une fois le rôle pris.

Quotas à connaître. 1 000 rôles et 1 500 politiques gérées par le client par compte, 10 politiques gérées attachées par identité. Ces valeurs par défaut se relèvent sur demande.

IAM, les fondamentaux, partie 3

Lire une politique JSON

Cinq éléments, toujours les mêmes : une fois la grille de lecture acquise, toute politique devient lisible.

- Structure d'un document de politique
- Effect, Action, Resource
- Condition et clés de contexte
- Principal et variables
- Trois exemples commentés
- Méthode de lecture et erreurs fréquentes

`{}` Une politique basée sur l'identité, avec deux déclarations

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ListerLeBucket",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::shop-images"
    },
    {
      "Sid": "LireEtEcrireLesObjets",
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:PutObject"],
      "Resource": "arn:aws:s3:::shop-images/*",
      "Condition": {
        "Bool": { "aws:SecureTransport": "true" }
      }
    }
  ]
}
```

Élément	Rôle
Version	Version du langage de politique. Toujours <code>2012-10-17</code> .
Statement	Liste des déclarations. Chacune est une règle indépendante.
Sid	Libellé facultatif de la déclaration, pour s'y retrouver.
Effect	<code>Allow</code> ou <code>Deny</code> .
Action	Les opérations concernées.
Resource	Les ressources concernées, par leur ARN.
Condition	Circonstances dans lesquelles la règle s'applique. Facultatif.

Définition : déclaration (statement)
Règle élémentaire d'une politique : un effet, appliqué à des actions, sur des ressources, éventuellement sous condition.

Schéma : les cinq éléments d'une déclaration

« Autoriser user7 à lire les objets du bucket shop-images, s'il se connecte depuis le bureau. »



Effect, Action et Resource sont obligatoires. Principal ne figure que dans les politiques de ressource.

Condition est facultative. En pointillés : les éléments qui ne sont pas toujours présents.

Lire une déclaration comme une phrase. « [Effect] à [Principal] de faire [Action] sur [Resource], si [Condition]. » Tant qu'on ne peut pas la dire en français, on ne l'a pas comprise.

Moyen mnémotechnique : PARC. Principal, Action, Ressource, Condition : les quatre éléments qui cernent une autorisation, auxquels s'ajoute l'effet.

Version

- Désigne la version du **langage**, pas celle de votre politique.
- Deux valeurs existent : `2012-10-17` (actuelle) et `2008-10-17` (ancienne).
- Sans cet élément, l'ancienne version s'applique, et les variables comme `${aws:username}` ne sont plus interprétées.
- Règle : toujours écrire `"Version": "2012-10-17"` en première ligne.

Statement

- Un objet unique ou, le plus souvent, une liste entre crochets.
- Les déclarations sont évaluées **toutes**, sans ordre : la première ne l'emporte pas sur la suivante.
- Une politique peut mêler des déclarations `Allow` et `Deny`.
- Regrouper dans une déclaration ce qui partage les mêmes ressources et les mêmes conditions.

Sid

- Identifiant de la déclaration, facultatif.
- Lettres et chiffres uniquement, sans espace, unique dans la politique.
- Sert de commentaire : JSON n'en admet pas d'autre.
- Bien choisi, il dit l'intention : `RefuserHorsEurope`, `LectureSeuleCatalogue`.

JSON strict. Guillemets doubles partout, virgule entre les éléments mais pas après le dernier, accolades et crochets appariés. Une faute de syntaxe fait rejeter la politique à l'enregistrement.

Effect : autoriser ou refuser

Définition : Effect

Élément obligatoire d'une déclaration, qui vaut `Allow` (autoriser) ou `Deny` (refuser). Il n'existe pas de troisième valeur.

	Allow	Deny
Effet	Accorde l'accès, si rien ne s'y oppose par ailleurs.	Interdit l'accès, quoi que disent les autres politiques.
Priorité	Cède devant tout Deny.	L'emporte toujours.
Usage	La règle générale : on construit les droits par des Allow.	L'exception : poser une interdiction ferme, un garde-fou.

Trois états possibles pour une requête

- **Autorisée explicitement** : une déclaration Allow s'applique, aucun Deny.
- **Refusée explicitement** : une déclaration Deny s'applique.
- **Refusée implicitement** : aucune déclaration ne s'applique. C'est l'état de départ de toute identité.

Quand écrire un Deny. Pour une règle qui ne doit souffrir aucune exception : interdire une région, exiger un second facteur pour les suppressions, protéger une ressource critique. Un Deny mal délimité bloque aussi les administrateurs.

Définition : Action
Élément qui liste les opérations d'API visées par la déclaration, sous la forme `service:Opération`.

Écriture	Signification
<code>"s3:GetObject"</code>	Une action précise.
<code>["s3:GetObject", "s3:PutObject"]</code>	Une liste d'actions.
<code>"s3:Get*"</code>	Toutes les actions de S3 qui commencent par Get.
<code>"s3:*"</code>	Toutes les actions de S3.
<code>"*"</code>	Toutes les actions de tous les services.
<code>"NotAction": "iam:*"</code>	Toutes les actions sauf celles d'IAM.

Le nom du service est en minuscules ; le nom de l'opération n'est pas sensible à la casse, mais on l'écrit comme dans la documentation.

Niveau d'accès	Exemples
List	<code>s3:ListBucket</code> , <code>ec2:DescribeInstances</code>
Read	<code>s3:GetObject</code> , <code>dynamodb:GetItem</code>
Write	<code>ec2:RunInstances</code> , <code>s3>DeleteObject</code>
Permissions management	<code>iam:AttachRolePolicy</code> , <code>s3:PutBucketPolicy</code>
Tagging	<code>ec2:CreateTags</code>

Où trouver les noms. La « Service Authorization Reference » liste, pour chaque service, toutes les actions, les ressources qu'elles acceptent et les clés de condition utilisables.

Attention aux jokers. `s3:*` inclut la suppression des buckets et la modification de leurs droits. Les actions de niveau « Permissions management » sont les plus sensibles.

Définition : Resource

Élément qui désigne, par leur ARN, les ressources auxquelles la déclaration s'applique.

Écriture	Désigne
<code>arn:aws:s3::shop-images</code>	Un bucket.
<code>arn:aws:s3::shop-images/*</code>	Tous ses objets.
<code>arn:aws:s3::shop-images/public/*</code>	Les objets d'un préfixe.
<code>arn:aws:ec2:eu-west-3:123456789012:instance/*</code>	Toutes les instances du compte à Paris.
<code>arn:aws:iam::123456789012:role/user7-*</code>	Les rôles dont le nom commence par user7-.
<code>*</code>	Toutes les ressources.

Règles

- Le joker `*` remplace n'importe quelle suite de caractères, `?` un seul caractère.
- L'ARN doit correspondre au **type de ressource attendu par l'action** : `s3:GetObject` porte sur un objet, `s3:ListBucket` sur un bucket.
- Certaines actions n'acceptent aucune ressource précise et exigent `"Resource": "*" : les listes (ec2:DescribeInstances, s3:ListAllMyBuckets) et certaines créations.`
- `NotResource` existe : tout sauf les ressources citées. À manier avec prudence.

Restreindre sans ARN. Quand une action n'accepte que `*`, on resserre par une condition : la région demandée, ou une étiquette de la ressource.

Schéma : un bucket et ses objets, deux ARN



arn:aws:s3:::shop-images

Le bucket lui-même. Actions : s3:ListBucket, s3:GetBucketLocation



arn:aws:s3:::shop-images/*

Les objets du bucket. Actions : s3:GetObject, s3:PutObject, s3:DeleteObject

Erreur classique

s3:GetObject accordé sur le bucket,
sans le /* final.

La politique est acceptée par IAM,
mais aucun objet n'est lisible :
l'action ne s'applique pas à cette ressource.

`{ }` La forme correcte : deux déclarations, deux types de ressources

```
{ "Version": "2012-10-17",  
  "Statement": [  
    { "Effect": "Allow", "Action": "s3:ListBucket",  
      "Resource": "arn:aws:s3:::shop-images" },  
    { "Effect": "Allow", "Action": ["s3:GetObject", "s3:PutObject"],  
      "Resource": "arn:aws:s3:::shop-images/*" } ] }
```

Le même principe vaut ailleurs. Une table DynamoDB et ses index, une file et ses messages, un groupe de logs et ses flux : toujours vérifier dans la documentation sur quel type de ressource porte chaque action. Quand une déclaration semble correcte mais n'a aucun effet, c'est souvent là qu'il faut chercher.

Condition : dans quelles circonstances

Définition : Condition

Élément facultatif qui restreint une déclaration aux requêtes dont le contexte satisfait un ou plusieurs tests.

```
{ } Forme générale : opérateur, clé, valeur

"Condition": {
  "IpAddress": {
    "aws:SourceIp": "203.0.113.0/24"
  }
}
```

Se lit : « si l'adresse IP source de la requête appartient à la plage 203.0.113.0/24 ».

Trois niveaux. L'**opérateur** dit comment comparer, la **clé** dit quelle donnée du contexte examiner, la **valeur** dit à quoi la comparer.

Opérateur	Compare
StringEquals, StringNotEquals	Texte, à l'identique.
StringLike, StringNotLike	Texte, avec jokers * et ?.
NumericLessThan, NumericEquals...	Nombres.
DateGreaterThan, DateLessThan	Dates.
Bool	Vrai ou faux.
IpAddress, NotIpAddress	Adresse ou plage IP.
ArnEquals, ArnLike	ARN.
Null	Présence ou absence de la clé.

Suffixe IfExists. StringEqualsIfExists applique le test seulement si la clé est présente dans la requête, et le considère comme réussi sinon.

Les clés de condition les plus utiles

Clé	Contient	Usage type
<code>aws:SourceIp</code>	Adresse IP d'où vient la requête.	N'autoriser que depuis le réseau de l'entreprise.
<code>aws:MultiFactorAuthPresent</code>	Vrai si un second facteur a été présenté.	Exiger le MFA pour les suppressions.
<code>aws:RequestedRegion</code>	Région visée par la requête.	Interdire tout ce qui sort d'Europe.
<code>aws:SecureTransport</code>	Vrai si la requête arrive en HTTPS.	Refuser les accès non chiffrés à un bucket.
<code>aws:CurrentTime</code>	Date et heure de la requête.	Accès limité dans le temps.
<code>aws:username</code>	Nom de l'utilisateur IAM.	Donner à chacun son propre dossier.
<code>aws:PrincipalTag/Clé</code>	Étiquette portée par le principal.	Contrôle d'accès par attributs.
<code>aws:ResourceTag/Clé</code>	Étiquette portée par la ressource.	N'agir que sur ses propres ressources.
<code>aws:RequestTag/Clé</code>	Étiquette passée dans la requête de création.	Imposer des étiquettes à la création.
<code>aws:PrincipalOrgID</code>	Identifiant de l'organisation du principal.	Ouvrir une ressource à toute l'organisation, et à elle seule.

Définition : clé de condition globale

Clé préfixée par `aws:`, disponible pour tous les services. Chaque service ajoute les siennes, préfixées par son nom : `s3:prefix`, `ec2:InstanceType`.

Clé absente. Si la clé n'existe pas dans la requête, le test échoue (sauf opérateurs négatifs et `IfExists`). `aws:MultiFactorAuthPresent` est par exemple absente des appels faits avec une clé d'accès permanente.

Schéma : la logique des conditions

```
"Condition": {  
  "StringEquals": { "aws:RequestedRegion": ["eu-west-3", "eu-central-1"] },  
  "Bool":          { "aws:MultiFactorAuthPresent": "true" }  
}
```

OU entre les valeurs d'une même clé
la région demandée est Paris ou Francfort

ET entre les clés et entre les opérateurs
la région convient et le second facteur est présent

Résultat : (région = Paris OU Francfort) ET (second facteur présent)

Si l'une des deux lignes est fausse, la déclaration ne s'applique pas.

Plusieurs valeurs

Une liste de valeurs pour une même clé se lit « l'une **ou** l'autre ». La région doit être Paris ou Francfort.

Plusieurs clés ou opérateurs

Ils se cumulent : « l'un **et** l'autre ». Tous les tests doivent réussir pour que la déclaration s'applique.

Opérateurs négatifs

Avec `StringNotEquals` et une liste, la valeur ne doit être égale à **aucun** des éléments : ni Paris, ni Francfort.

Définition : Principal

Élément qui désigne à qui s'applique une déclaration. Il figure uniquement dans les politiques basées sur la ressource et dans les trust policies.

Écriture	Désigne
<code>{"AWS": "arn:aws:iam::111122223333:root"}</code>	Le compte entier, qui délègue ensuite à ses identités.
<code>{"AWS": "arn:aws:iam::111122223333:role/Audit"}</code>	Un rôle précis.
<code>{"Service": "lambda.amazonaws.com"}</code>	Un service AWS.
<code>{"Federated": "..."} "..."</code>	Un fournisseur d'identité externe. Tout le monde, anonymes compris.

`"Principal": "*"` avec `Allow` et sans condition rend la ressource publique. C'est l'origine de la plupart des fuites de buckets S3.

Définition : variable de politique

Marqueur de la forme `${clé}`, remplacé au moment de l'évaluation par la valeur de la clé dans la requête.

```
} Chaque utilisateur accède à son seul dossier

{ "Version": "2012-10-17",
  "Statement": [ {
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource":
      "arn:aws:s3:::shop-partage/home/${aws:username}/*"
  } ] }
```

Intérêt. Une seule politique, attachée au groupe, sert pour quinze apprenants : pour `user7`, la ressource devient `.../home/user7/*`. Sans variable, il faudrait quinze politiques.

Exemple commenté 1 : lecture seule sur un bucket

`{}` Politique d'identité

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VoirLaListeDesBuckets",
      "Effect": "Allow",
      "Action": "s3:ListAllMyBuckets",
      "Resource": "*"
    },
    {
      "Sid": "ListerLeContenu",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::shop-images"
    },
    {
      "Sid": "LireLesObjets",
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::shop-images/*"
    }
  ]
}
```

Déclaration

Lecture

VoirLaListeDesBuckets

Autoriser à lister les buckets du compte. L'action n'accepte pas de ressource précise, d'où `*`. Sans elle, la console S3 affiche une erreur.

ListerLeContenu

Autoriser à voir les noms des objets du bucket shop-images. La ressource est le bucket.

LireLesObjets

Autoriser à télécharger les objets. La ressource est l'ensemble des objets.

Ce que le porteur ne peut pas faire

- Écrire ou supprimer un objet : aucune déclaration ne l'autorise.
- Lire un autre bucket : il le voit dans la liste, mais l'ouverture est refusée.
- Toute action hors S3.

Tout ce qui n'est pas écrit est refusé : on lit une politique autant par ce qu'elle ne dit pas.

Exemple commenté 2 : agir sur ses seules instances

`{}` Contrôle d'accès par étiquette

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VoirToutesLesInstances",
      "Effect": "Allow",
      "Action": "ec2:Describe*",
      "Resource": "*"
    },
    {
      "Sid": "PiloterSesPropresInstances",
      "Effect": "Allow",
      "Action": [
        "ec2:StartInstances",
        "ec2:StopInstances",
        "ec2:RebootInstances"
      ],
      "Resource": "arn:aws:ec2:eu-west-3:123456789012:instance/*",
      "Condition": {
        "StringEquals": {
          "aws:ResourceTag/Owner": "${aws:username}"
        }
      }
    }
  ]
}
```

Déclaration

Lecture

VoirToutesLesInstances

Autoriser toutes les actions de consultation d'EC2, sur tout. Les actions Describe n'acceptent pas de ressource précise.

PiloterSesPropresInstances

Autoriser à démarrer, arrêter et redémarrer les instances de Paris, **si** leur étiquette Owner est égale au nom de l'utilisateur qui fait la requête.

Définition : ABAC (contrôle d'accès par attributs)

Modèle dans lequel l'autorisation dépend de la correspondance entre des attributs du demandeur et des attributs de la ressource, ici des étiquettes.

Résultat. user7 peut arrêter l'instance étiquetée `owner = user7`, pas celle de user8. La même politique sert à tous. C'est l'usage des étiquettes annoncé au chapitre « Premiers pas dans la console ».

Encore faut-il que l'utilisateur ne puisse pas modifier l'étiquette Owner d'une instance : `ec2:CreateTags` doit rester sous contrôle.

Exemple commenté 3 : deux garde-fous par un Deny

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "RefuserHorsEurope",
      "Effect": "Deny",
      "NotAction": ["iam:*", "sts:*", "cloudfront:*", "route53:*", "support:*"],
      "Resource": "*",
      "Condition": {
        "StringNotEquals": {
          "aws:RequestedRegion": ["eu-west-3", "eu-central-1"]
        }
      }
    },
    {
      "Sid": "SupprimerSeulementAvecMFA",
      "Effect": "Deny",
      "Action": ["ec2:TerminateInstances", "rds>DeleteDBInstance"],
      "Resource": "*",
      "Condition": {
        "BoolIfExists": {
          "aws:MultiFactorAuthPresent": "false"
        }
      }
    }
  ]
}
```

Déclaration	Lecture
RefuserHorsEurope	Refuser toute action, sauf celles des services mondiaux listés, si la région visée n'est ni Paris ni Francfort.
SupprimerSeulementAvec MFA	Refuser la suppression d'instances et de bases si aucun second facteur n'a été présenté.

Points à remarquer

- Cette politique **n'accorde rien** : elle retranche. Elle s'attache en plus d'une politique qui autorise.
- `NotAction` avec `Deny` : « refuser tout sauf ». Les services mondiaux sont exclus, car leurs appels partent vers us-east-1.
- `BoolIfExists` : le refus s'applique aussi quand la clé MFA est absente, comme avec une clé d'accès permanente.

Un Deny l'emporte sur tout Allow : même un administrateur porteur de cette politique est bloqué hors d'Europe.

#	Question	Où regarder
1	Autorise-t-elle ou refuse-t-elle ?	Effect
2	Quelles actions, de quel service ? Y a-t-il des jokers ?	Action ou NotAction
3	Sur quelles ressources exactement ?	Resource : région, compte, type, nom
4	Dans quelles circonstances ?	Condition
5	À qui cela s'applique-t-il ?	L'identité qui porte la politique, ou Principal

Puis trois vérifications

- L'ARN correspond-il au type de ressource attendu par chaque action ?
- Un joker ouvre-t-il plus que voulu : `*` dans Action, dans Resource, dans Principal ?
- Qu'est-ce qui n'est **pas** écrit, donc refusé ?

Signaux d'alerte. `"Action": "*" avec "Resource": "*" : droits d'administrateur.` `"Principal": "*" : accès public.` `iam:*` ou `iam:PassRole` sur `*` : possibilité de s'accorder des droits.

Outil. L'éditeur visuel de la console affiche chaque déclaration sous forme de résumé : services, niveaux d'accès, ressources, conditions. Il signale aussi les erreurs et les avertissements de l'analyseur de politiques.

Erreurs fréquentes dans l'écriture d'une politique

Erreur	Symptôme	Correction
ARN du bucket sans <code>/*</code> pour une action sur les objets	Politique acceptée, accès refusé.	Deux déclarations : bucket, puis objets.
Ressource précise sur une action qui n'en accepte pas	L'action reste refusée.	<code>"Resource": "*" ,</code> resserré par une condition.
Virgule en trop, guillemet manquant	« Malformed policy document » à l'enregistrement.	Valider le JSON ; utiliser l'éditeur de la console.
Version omise	Les variables <code>\${...}</code> sont prises pour du texte.	Toujours <code>2012-10-17</code> .
Faute dans un nom d'action	Avertissement, puis action jamais autorisée.	Copier le nom depuis la documentation.
Champ Principal dans une politique d'identité	Politique rejetée.	Le supprimer : le principal est le porteur.
Condition sur une clé que la requête ne contient pas	Déclaration jamais appliquée.	Vérifier les clés acceptées par l'action ; IfExists si besoin.
Deny trop large	Tout le monde est bloqué, administrateurs compris.	Délimiter par action, ressource et condition ; tester d'abord.
Région ou numéro de compte erronés dans l'ARN	Accès refusé sans raison apparente.	Copier l'ARN depuis la console.

IAM, les fondamentaux, partie 4

L'évaluation des droits

Refus par défaut, refus explicite, moindre privilège : comment AWS décide, et comment accorder juste ce qu'il faut.

- Les trois règles
- La logique d'évaluation
- Cumul des politiques
- Même compte, comptes différents
- Les garde-fous
- Le moindre privilège et ses outils

Les trois règles de l'évaluation

Définition : refus par défaut (refus implicite)

En l'absence de toute déclaration qui autorise une action, celle-ci est refusée. C'est l'état initial de toute identité.

Définition : autorisation explicite

Déclaration `Allow` qui correspond à la requête. Elle lève le refus par défaut.

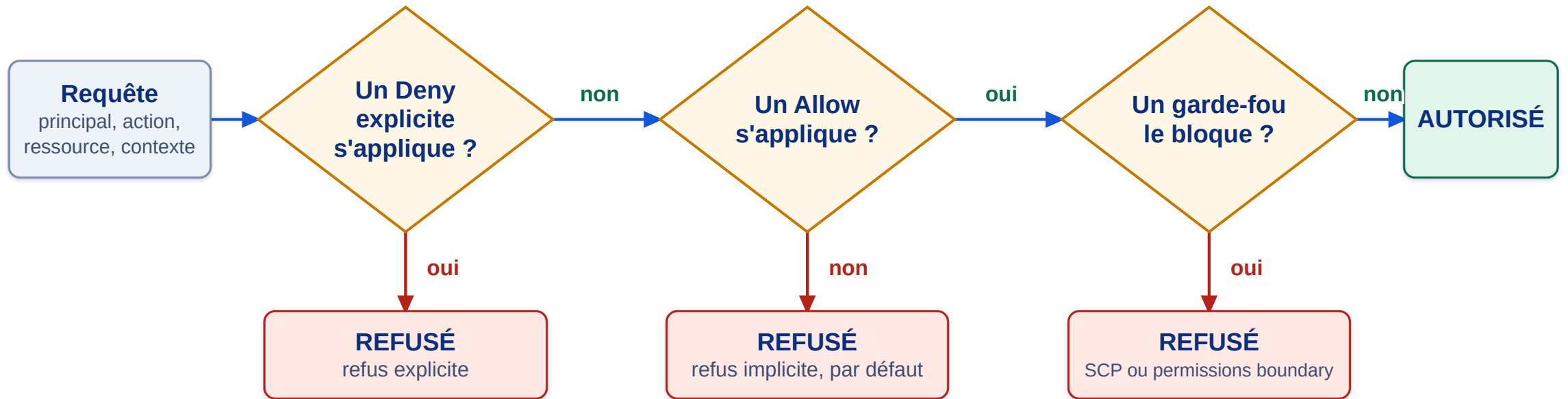
Définition : refus explicite

Déclaration `Deny` qui correspond à la requête. Elle l'emporte sur toutes les autorisations, sans exception.

#	Règle	Conséquence
1	Tout est refusé au départ.	Un oubli se traduit par un accès refusé, jamais par un accès ouvert. L'erreur par omission est sans danger.
2	Un Allow autorise.	Les droits se construisent en ajoutant des autorisations, de n'importe quelle politique applicable.
3	Un Deny gagne toujours.	Aucune autorisation, même d'administrateur, ne peut contourner un refus explicite.

Pas d'ordre, pas de priorité entre politiques. AWS ne lit pas les politiques l'une après l'autre en s'arrêtant à la première qui correspond, comme le ferait un pare-feu. Il les rassemble toutes, puis applique ces trois règles à l'ensemble.

Schéma : la logique d'évaluation



Premier test

AWS cherche un Deny applicable dans **toutes** les politiques concernées. S'il en trouve un, la décision est prise : refusé.

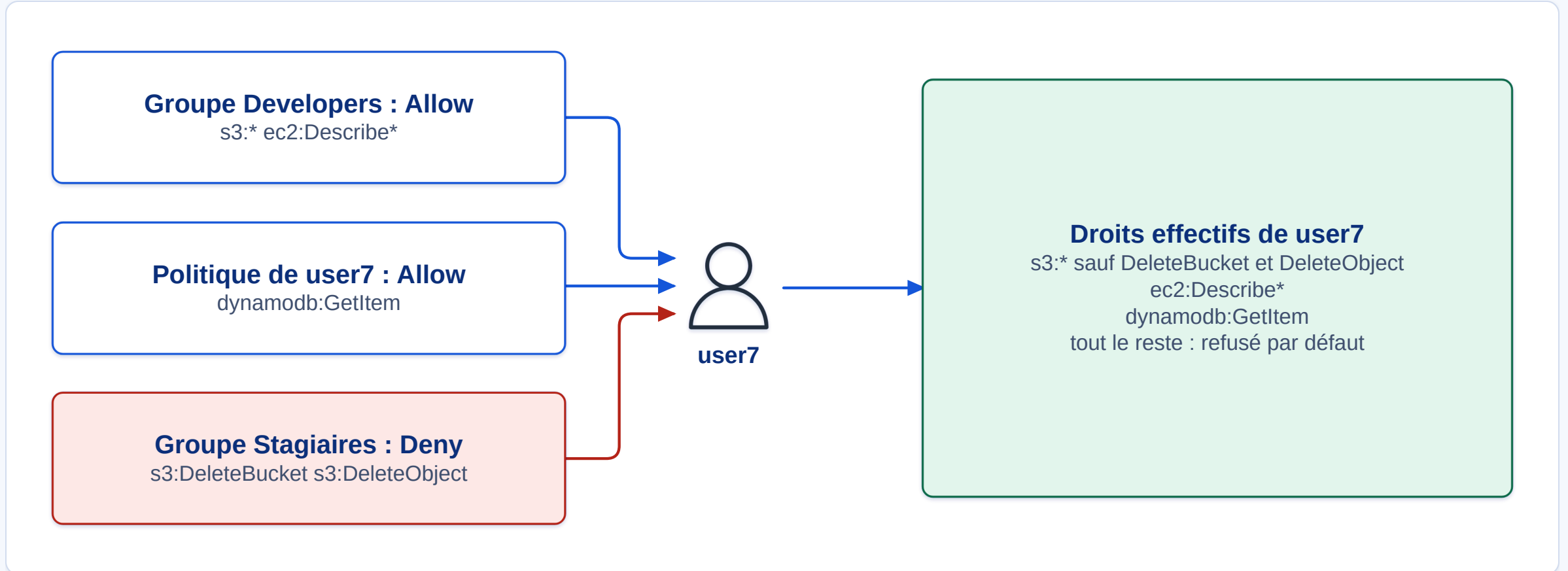
Deuxième test

Il cherche un Allow dans les politiques d'identité ou de ressource. S'il n'en trouve aucun, c'est le refus par défaut.

Troisième test

Si des garde-fous existent (SCP de l'organisation, permissions boundary), l'action doit aussi y être permise.

Schéma : cumul des autorisations, retrait des refus



Lecture. user7 reçoit l'**union** de toutes les autorisations de ses groupes et de ses propres politiques. Le refus posé par le groupe Stagiaires retranche deux actions de cet ensemble, bien que s3:* les ait accordées ailleurs.

Conséquence. On ne retire pas un droit en « ne le donnant pas » dans un groupe si un autre groupe le donne. Pour interdire à coup sûr, il faut un Deny, ou sortir l'utilisateur du groupe qui autorise.

#	Politiques en présence	Requête	Résultat	Pourquoi
1	Aucune politique attachée.	s3:ListAllMyBuckets	Refusé	Refus par défaut.
2	Allow s3:GetObject sur shop-images/*.	s3:PutObject dans shop-images	Refusé	Aucun Allow pour cette action.
3	Groupe A : Allow ec2:* . Groupe B : Deny ec2:TerminateInstances .	ec2:TerminateInstances	Refusé	Le Deny l'emporte.
4	Groupe A : Allow ec2:* . Groupe B : Deny ec2:TerminateInstances .	ec2:StopInstances	Autorisé	Allow, aucun Deny pour cette action.
5	Allow s3:* sur *, avec la condition aws:RequestedRegion = eu-west-3 .	s3:CreateBucket à Francfort	Refusé	La condition est fausse : l'Allow ne s'applique pas.

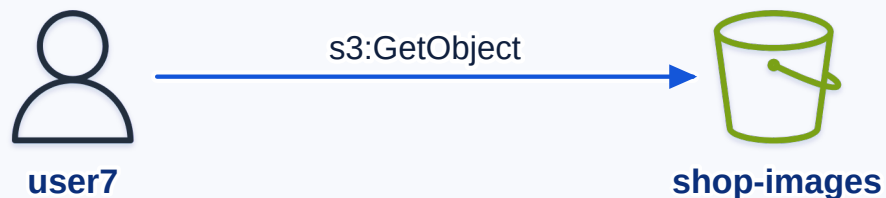
Méthode

1. Identifier le principal, l'action, la ressource et le contexte de la requête.
2. Rassembler toutes les politiques qui s'appliquent au principal et à la ressource.
3. Un Deny correspond ? Refusé.
4. Un Allow correspond ? Autorisé, sauf garde-fou.
5. Sinon : refusé par défaut.

« **Correspond** » signifie que l'action, la ressource **et** la condition de la déclaration sont toutes satisfaites par la requête. Une déclaration dont la condition est fausse est simplement ignorée, qu'elle soit Allow ou Deny.

Schéma : même compte ou comptes différents

Même compte : une seule autorisation suffit

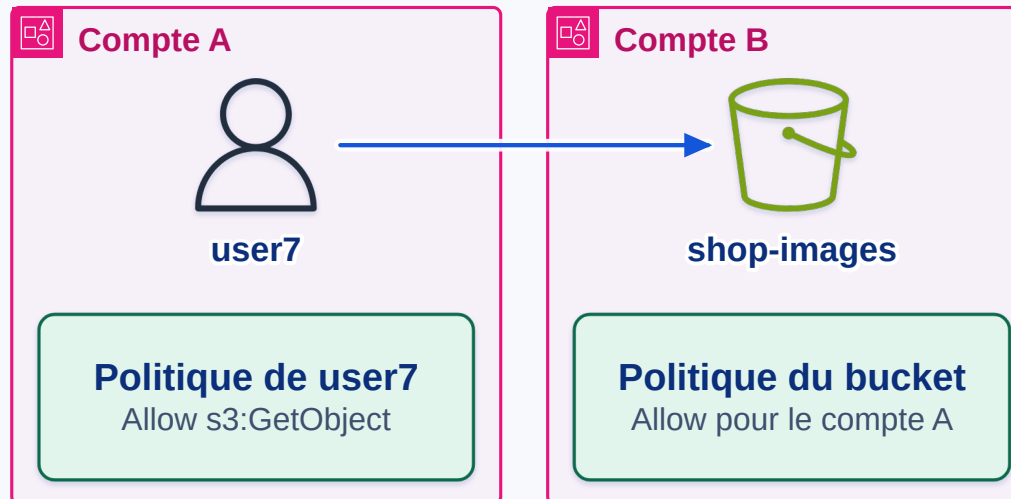


Politique de user7
aucune autorisation sur S3

Politique du bucket
Allow pour user7

Résultat : autorisé

Deux comptes : il faut les deux autorisations

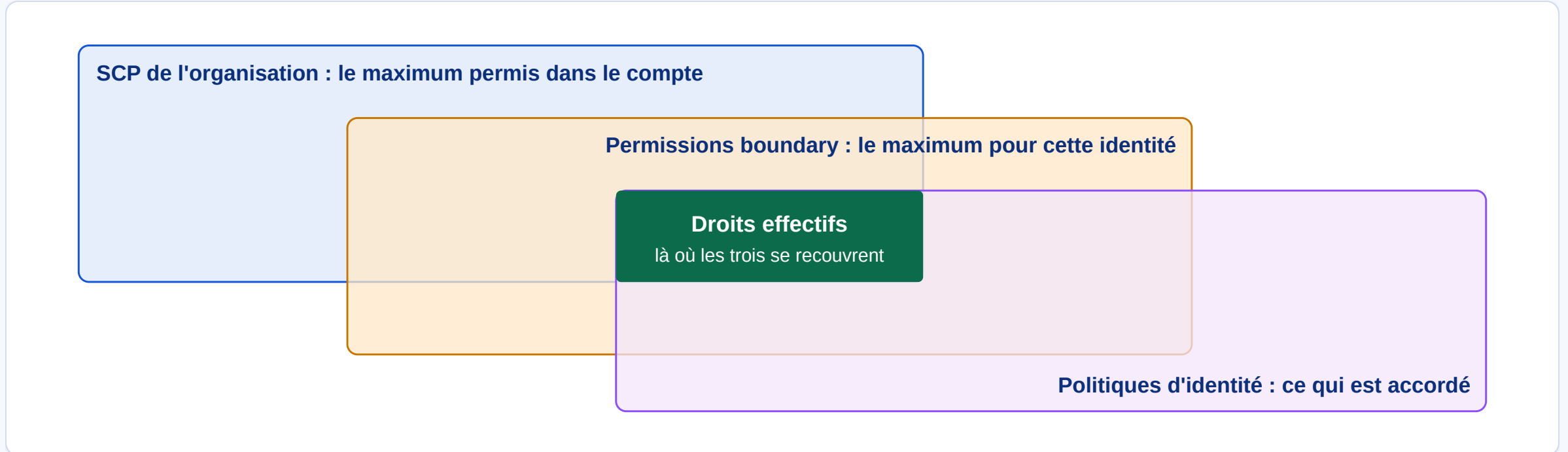


Résultat : autorisé seulement si les deux disent oui

Dans un même compte, politique d'identité et politique de ressource s'additionnent : l'une ou l'autre suffit à autoriser (hors Deny). C'est pourquoi un bucket peut ouvrir un accès à un utilisateur sans qu'on touche à cet utilisateur.

Entre deux comptes, chacun garde la maîtrise de son côté : le compte de l'identité doit l'autoriser à sortir, le compte de la ressource doit l'autoriser à entrer. Aucun des deux ne peut décider seul.

Schéma : les garde-fous délimitent, ils n'accordent pas



Définition : permissions boundary

Politique gérée désignée comme plafond d'un utilisateur ou d'un rôle : quelles que soient ses autres politiques, il ne peut dépasser ce qu'elle permet.

Définition : SCP (politique de contrôle des services)

Règle d'AWS Organizations qui fixe le maximum de ce que les identités d'un compte peuvent faire, utilisateur racine compris.

Usage type de la boundary. Déléguer la création de rôles sans risque : « tu peux créer des rôles, à condition qu'ils portent cette limite ». Un rôle ainsi créé ne peut jamais avoir plus de droits que son créateur, même si on lui attache `AdministratorAccess`.

#	Ce qu'AWS examine	Question posée	Si la réponse est défavorable
1	Tous les refus explicites	Une déclaration Deny correspond-elle, où que ce soit ?	Refusé, fin de l'évaluation.
2	SCP et RCP de l'organisation	L'action est-elle permise pour ce compte ?	Refusé.
3	Politique de la ressource	Autorise-t-elle ce principal ?	On continue (même compte) ; refusé (autre compte).
4	Politiques d'identité	Une déclaration Allow correspond-elle ?	Refusé par défaut, sauf autorisation à l'étape 3.
5	Permissions boundary	L'action est-elle dans la limite de l'identité ?	Refusé.
6	Politique de session	La session en cours a-t-elle été restreinte ?	Refusé.

Dans un compte simple, sans organisation ni boundary, seules les étapes 1, 3 et 4 existent. C'est le cas de la plupart des exercices de cette formation.

Deux façons d'être refusé. Par un Deny écrit quelque part (explicite), ou parce qu'aucune étape n'a autorisé (implicite). Le message d'erreur précise laquelle : c'est le premier indice d'un diagnostic.

Définition : moindre privilège

Principe selon lequel une identité ne reçoit que les droits strictement nécessaires à sa tâche : les seules actions utiles, sur les seules ressources concernées, dans les seules conditions prévues.

Pourquoi

- **Limiter les dégâts d'une compromission** : des identifiants volés ne donnent que les droits de leur titulaire.
- **Limiter les dégâts d'une erreur** : une commande lancée dans le mauvais terminal ne peut pas supprimer ce qu'elle n'a pas le droit de toucher.
- **Rendre les droits lisibles** : une politique étroite dit ce que fait l'application.
- **Satisfaire les audits** : c'est une exigence de tous les référentiels de sécurité.

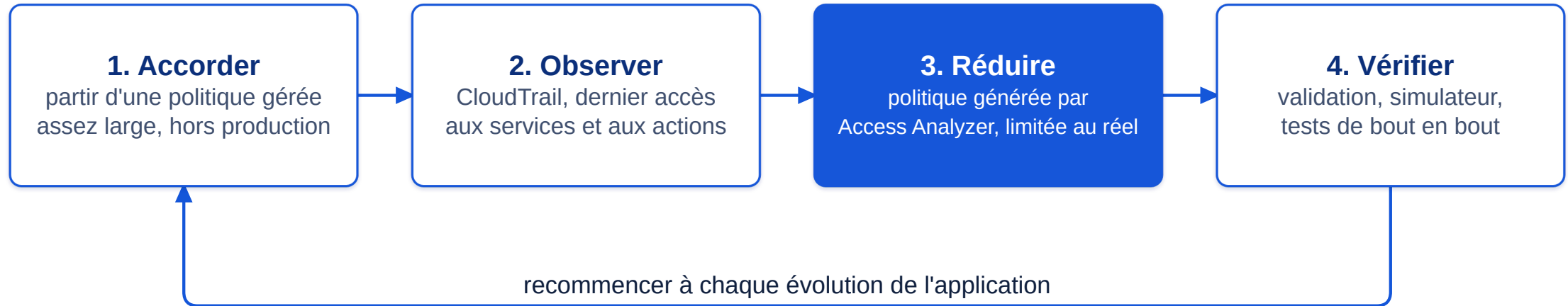
Trois dimensions à resserrer

- **Les actions** : `s3:GetObject` plutôt que `s3:*`.
- **Les ressources** : un bucket nommé plutôt que `*`.
- **Les conditions** : une région, une étiquette, un second facteur, une plage horaire.

À quoi s'ajoute la **durée** : des identifiants temporaires plutôt que permanents.

L'équilibre à trouver. Des droits trop larges exposent ; des droits trop étroits bloquent le travail et poussent à tout rouvrir « pour que ça marche ». Le moindre privilège est un objectif que l'on approche par étapes, pas un réglage que l'on trouve du premier coup.

Schéma : approcher le moindre privilège par itérations



Démarche recommandée par AWS

Commencer par une politique gérée par AWS adaptée à la fonction, dans un environnement sans enjeu. Observer ce qui est réellement utilisé pendant quelques semaines. Remplacer ensuite par une politique sur mesure limitée à cet usage.

Les données qui guident

Dernier accès : IAM indique pour chaque identité les services et les actions utilisés, et depuis quand. **CloudTrail** : le détail des appels. Un service jamais utilisé en 90 jours peut être retiré de la politique.

Les outils pour vérifier et réduire les droits



Access Analyzer : validation

Vérifie une politique pendant sa rédaction : erreurs, avertissements de sécurité, suggestions.



Access Analyzer : génération

Produit une politique à partir de l'activité réelle d'une identité relevée dans CloudTrail.



Access Analyzer : accès externes

Signale les ressources ouvertes à un autre compte ou au public.



Simulateur de politiques

Teste une action et une ressource pour une identité, et montre quelle déclaration décide.



CloudTrail

Montre les appels réellement effectués, et ceux qui ont été refusés.



Dernier accès et rapport d'identifiants

Repèrent les droits et les identifiants inutilisés.

\$ Simuler en ligne de commande : user7 peut-il lire un objet, supprimer un bucket ?

```
aws iam simulate-principal-policy \
  --policy-source-arn arn:aws:iam::123456789012:user/user7 \
  --action-names s3:GetObject s3:DeleteBucket \
  --resource-arns "arn:aws:s3:::shop-images/logo.png" \
  --query 'EvaluationResults[][EvalActionName,EvalDecision]' --output table
```

Le résultat vaut `allowed`, `implicitDeny` ou `explicitDeny` : les trois états vus plus haut.

txt Refus implicite : personne n'a autorisé

```
An error occurred (AccessDenied) when calling the GetObject operation:
User: arn:aws:iam::123456789012:user/user7 is not authorized to perform: s3:GetObject
on resource: "arn:aws:s3:::shop-images/logo.png"
because no identity-based policy allows the s3:GetObject action
```

txt Refus explicite : un Deny s'applique

```
User: arn:aws:sts::123456789012:assumed-role/RoleApprenants/user7 is not authorized to perform:
iam:CreateUser on resource: arn:aws:iam::123456789012:user/test
with an explicit deny in an identity-based policy
```

Élément du message	Ce qu'il apprend
User	Le principal réel de la requête. Est-ce bien l'identité attendue ? Ici, dans le second cas, la session du rôle et non l'utilisateur.
perform	L'action exacte à autoriser : c'est le nom à mettre dans <code>Action</code> .
on resource	L'ARN exact à mettre dans <code>Resource</code> .
because / with	Refus implicite (il manque un Allow) ou explicite (un Deny à trouver), et dans quel type de politique.

Moindre privilège : les erreurs à éviter

Pratique	Risque	À la place
"Action": "*" et "Resource": "*" pour une application	Une faille dans l'application donne le contrôle du compte.	Les seules actions utilisées, sur les seules ressources de l'application.
AdministratorAccess donné à toute l'équipe	Toute erreur peut toucher la production.	Droits courants limités ; rôle d'administration pris à la demande.
Droits IAM accordés largement (iam:*)	L'identité peut s'accorder elle-même tous les droits.	Séparer la gestion d'IAM ; permissions boundary.
Politique copiée d'un forum sans la lire	Droits inconnus, souvent trop larges.	Lire chaque déclaration avec la méthode en cinq questions.
Droits jamais revus	Accumulation au fil des projets et des changements de poste.	Revue périodique d'après le dernier accès.
Élargir dès le premier refus	Le droit temporaire devient définitif.	Lire le message, ajouter l'action et la ressource exactes.
Un seul rôle pour tous les services	La compromission d'un service expose tous les autres.	Un rôle par service ou par fonction.

Définition : élévation de privilèges

Fait, pour une identité, d'obtenir des droits supérieurs à ceux qu'on lui a accordés, en exploitant un droit mal délimité : créer une politique, attacher une politique, confier un rôle à un service.

IAM, les fondamentaux, partie 5

Politiques gérées et politiques personnalisées

Trois façons de ranger une politique d'identité, selon qui l'écrit et qui la porte.

- Les trois sortes de politiques d'identité
- Politiques gérées par AWS
- Politiques gérées par le client
- Versions et retour arrière
- Politiques en ligne
- Comment choisir

Types de politiques

Trois sortes de politiques d'identité

Définition : politique gérée par AWS

Politique écrite, publiée et tenue à jour par AWS, que l'on attache telle quelle. Son ARN commence par `arn:aws:iam::aws:policy/`.

Définition : politique gérée par le client

Politique autonome que vous créez dans votre compte, que vous modifiez et que vous attachez à autant d'identités que nécessaire.

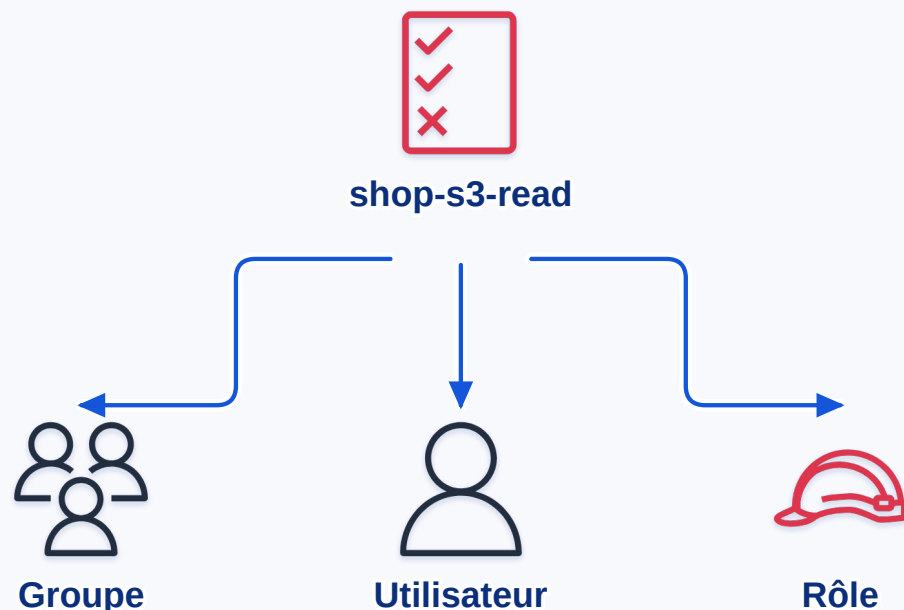
Définition : politique en ligne (inline)

Politique écrite directement dans un utilisateur, un groupe ou un rôle, dont elle fait partie et avec lequel elle disparaît.

	Gérée par AWS	Gérée par le client	En ligne
Écrite par	AWS	Vous	Vous
Modifiable	Non	Oui, avec versions	Oui, sans versions
Réutilisable	Oui, dans tous les comptes	Oui, dans votre compte	Non : une seule identité
Possède un ARN	Oui	Oui	Non
Précision	Générale, souvent large	Ajustée à vos ressources	Ajustée à une identité

Schéma : politique gérée et politique en ligne

Politique gérée : un objet autonome, réutilisable



Possède son ARN. Une modification s'applique à tous les porteurs.

Politique en ligne : incluse dans une seule identité

Rôle shop-export



Politique en ligne
écrite dans le rôle

Pas d'ARN, pas de versions, non réutilisable.

Disparaît avec l'identité qui la contient.

Lecture. La politique gérée vit sa propre vie : on la crée, on la versionne, on l'attache et on la détache. La politique en ligne est une propriété de l'identité qui la contient. La première se mutualise, la seconde garantit qu'un droit reste lié à une identité et à elle seule.

Caractéristiques

- Plus d'un millier, couvrant chaque service et les fonctions courantes.
- Mises à jour par AWS quand un service évolue : une nouvelle action de lecture est ajoutée d'office aux politiques de lecture seule.
- Non modifiables : on ne peut ni les restreindre ni les étendre.
- Écrites pour tous les clients, donc presque toujours avec `"Resource": "*" .`

Limite. Elles ne connaissent pas vos ressources :

`AmazonS3FullAccess` donne tous les droits sur **tous** vos buckets. Pratiques pour démarrer, rarement conformes au moindre privilège.

Famille	Exemples
Accès complet	<code>AdministratorAccess</code> , <code>AmazonEC2FullAccess</code>
Lecture seule	<code>ReadOnlyAccess</code> , <code>AmazonS3ReadOnlyAccess</code>
Par fonction (job functions)	<code>PowerUserAccess</code> , <code>Billing</code> , <code>SecurityAudit</code> , <code>DatabaseAdministrator</code> , <code>NetworkAdministrator</code>
Pour les rôles de service	<code>AWSLambdaBasicExecutionRole</code> , <code>AmazonECSTaskExecutionRolePolicy</code>
Libre-service	<code>IAMUserChangePassword</code>

Reconnaître leur ARN. Le numéro de compte est remplacé par `aws` : `arn:aws:iam::aws:policy/ReadOnlyAccess` . Dans la console, elles portent une icône AWS orange.

Quatre politiques gérées par AWS à connaître

Politique	Ce qu'elle contient	Usage et précaution
AdministratorAccess	Allow sur "Action": "*" et "Resource": "*" .	Administrateurs du compte, en nombre réduit. Donne aussi la gestion d'IAM, donc tous les pouvoirs sauf ceux de la racine.
PowerUserAccess	Allow avec NotAction sur iam:*, organizations:* et account:* : tout, sauf la gestion des identités et du compte.	Développeurs qui créent des ressources sans gérer les accès. C'est la base de RoleApprenants .
ReadOnlyAccess	Les actions de consultation de tous les services.	Audit, support, découverte. Attention : inclut la lecture du contenu des buckets et des tables.
ViewOnlyAccess	Lister et décrire les ressources, sans lire les données qu'elles contiennent.	Variante plus prudente de la précédente.

`{}` Le cœur de PowerUserAccess : Allow combiné à NotAction

```
{ "Effect": "Allow",
  "NotAction": [ "iam:*", "organizations:*", "account:*" ],
  "Resource": "*" }
```

Lecture. « Autoriser tout ce qui n'est pas IAM, Organizations ou Account. » Ce n'est pas un refus : si une autre politique autorise iam:CreateRole , l'action reste permise. Pour l'interdire fermement, il faudrait un Deny .

Quand en écrire

- Dès que l'on veut désigner **ses propres ressources** : ce bucket, ces tables, ce préfixe de noms.
- Pour combiner des droits de plusieurs services correspondant à une fonction de l'entreprise.
- Pour poser un garde-fou commun, attaché à plusieurs groupes.

Bonnes pratiques

- Un nom qui dit l'intention : `shop-catalogue-lecture`.
- Une politique par besoin cohérent, plutôt qu'une politique fourre-tout.
- Une description, et des `Sid` explicites.
- Stockée dans un dépôt Git et déployée par un outil d'infrastructure as code.

Limite	Valeur
Taille d'une politique gérée	6 144 caractères
Versions conservées	5
Politiques gérées par compte	1 500
Politiques attachées à une identité	10

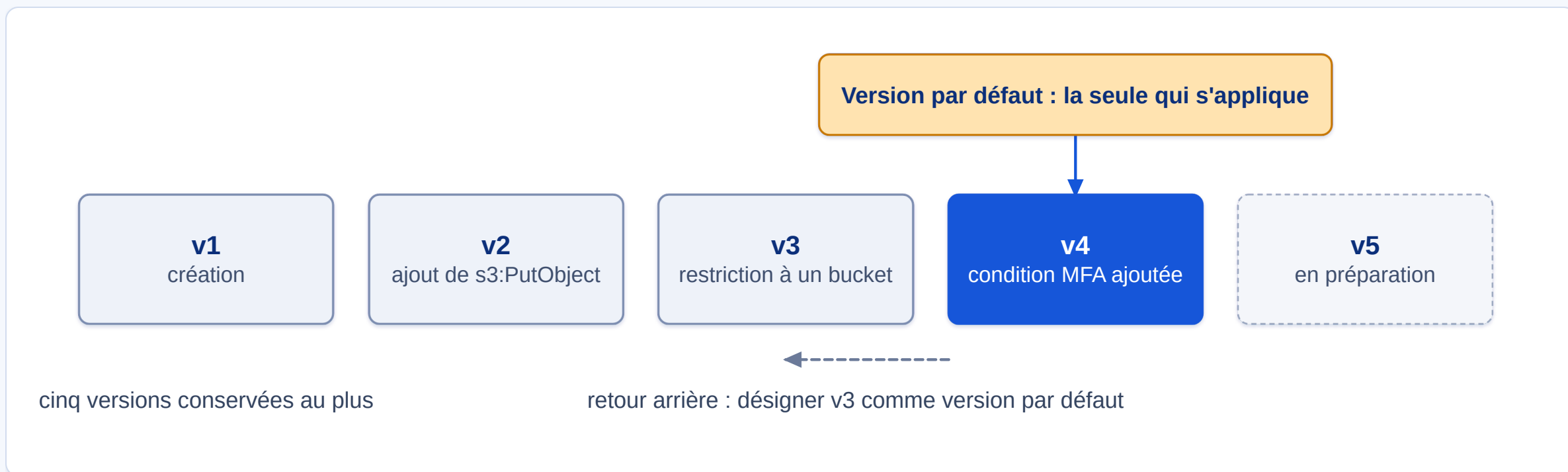
\$ Créer et attacher une politique gérée par le client

```
aws iam create-policy --policy-name shop-catalogue-lecture \  
  --policy-document file://catalogue-lecture.json \  
  --description "Lecture du bucket shop-images"
```

```
aws iam attach-group-policy --group-name Developers \  
  --policy-arn \  
  arn:aws:iam::123456789012:policy/shop-catalogue-lecture
```

Prérequis de l'exemple : un fichier `catalogue-lecture.json` contenant la politique, et un groupe `Developers` existant.

Schéma : les versions d'une politique gérée



Fonctionnement

Chaque modification crée une nouvelle version. Une seule est active : la **version par défaut**. IAM en garde cinq ; au-delà, il faut en supprimer une avant d'en créer une autre.

Intérêt

Une modification qui casse un service se corrige en une commande : on redésigne la version précédente comme version par défaut. Tous les porteurs de la politique retrouvent aussitôt leurs anciens droits.

Quand une politique en ligne se justifie

- Le droit ne doit **jamais** être attaché par erreur à une autre identité.
- Le droit n'a de sens que pour cette identité : la politique propre à un rôle de service unique.
- On veut que la suppression de l'identité emporte ses droits, sans objet orphelin.

Ses inconvénients

- Pas de versions, donc pas de retour arrière.
- Pas de vue d'ensemble : il faut ouvrir chaque identité pour les trouver.
- Duplication dès que deux identités ont le même besoin.

Situation	Choix
Démarrer vite, besoin standard	Gérée par AWS
Rôle de service d'un service AWS	Gérée par AWS prévue pour ce service
Droits sur vos ressources, pour plusieurs identités	Gérée par le client
Production, moindre privilège	Gérée par le client
Droit strictement lié à une identité	En ligne

Recommandation d'AWS. Préférer les politiques gérées aux politiques en ligne ; commencer par les politiques gérées par AWS, puis les remplacer par des politiques gérées par le client à mesure que les besoins se précisent.

Dans la console, l'onglet **Permissions** d'une identité distingue les trois : « AWS managed », « Customer managed », « Customer inline ».

`{ }` Politique en ligne du rôle (extrait), en plus de PowerUserAccess

```
{ "Version": "2012-10-17",
  "Statement": [
    { "Sid": "LectureIam",
      "Effect": "Allow",
      "Action": ["iam:Get*", "iam:List*"],
      "Resource": "*" },
    { "Sid": "GestionDesRoles",
      "Effect": "Allow",
      "Action": ["iam:CreateRole", "iam:DeleteRole",
                 "iam:AttachRolePolicy", "iam:PassRole"],
      "Resource": "*" },
    { "Sid": "PasToucheAuRoleApprenants",
      "Effect": "Deny",
      "NotAction": ["iam:Get*", "iam:List*"],
      "Resource":
        "arn:aws:iam::123456789012:role/RoleApprenants" },
    { "Sid": "PasDeFacturation",
      "Effect": "Deny",
      "Action": ["billing:*", "ce:*", "budgets:*"],
      "Resource": "*" }
  ] }
```

Élément	Effet
PowerUserAccess (gérée par AWS)	Tous les services, sauf IAM, Organizations et Account.
LectureIam	Rend IAM consultable, ce que PowerUserAccess n'accordait pas.
GestionDesRoles	Ajoute la création de rôles, nécessaire aux travaux pratiques.
PasToucheAuRoleApprenants	Refus explicite : le rôle ne peut pas se modifier lui-même.
PasDeFacturation	Refus explicite, alors que PowerUserAccess l'autorisait.

Ce que l'on retrouve. Une politique gérée par AWS comme socle, des Allow qui complètent, des Deny qui retranchent. Et ce qui n'est écrit nulle part reste refusé : créer un utilisateur, par exemple.

Question à se poser. Ce rôle peut créer d'autres rôles et leur attacher des politiques : rien ne l'empêche d'en créer un plus puissant que lui. C'est le cas d'école de la permissions boundary.

Ce qu'il faut retenir

- IAM authentifie, puis autorise, chaque appel d'API.
- Utilisateur : identifiants permanents. Groupe : ensemble d'utilisateurs. Rôle : droits prêtés, identifiants temporaires. Politique : le document qui décrit les droits.
- Une déclaration : Effect, Action, Resource, et parfois Condition ou Principal.
- Tout est refusé par défaut ; un Allow autorise ; un Deny l'emporte toujours.
- Les autorisations de toutes les politiques se cumulent.
- Moindre privilège : resserrer actions, ressources et conditions, par itérations.
- Gérée par AWS pour démarrer, gérée par le client pour préciser, en ligne pour l'exception.

1. Quelle différence entre authentification et autorisation ?

2. Un groupe peut-il être cité comme Principal ?

3. Quels sont les deux documents qui définissent un rôle ?

4. Pourquoi faut-il deux ARN pour lire un bucket S3 ?

5. Un groupe autorise s3:*, un autre refuse s3:DeleteBucket. Résultat ?

6. Comment se lisent plusieurs valeurs d'une même clé de condition ?

7. Que permet PowerUserAccess, et que ne permet-elle pas ?

Cliquer sur une question pour afficher la réponse.

La CLI et les rôles, partie 1

La CLI : installation et connexion

Piloter AWS depuis son terminal, sans stocker de clé permanente sur le poste.

- Ce qu'est l'AWS CLI
- Installation
- Les façons de s'authentifier
- aws login
- Profils, région, format de sortie
- Ordre de priorité des réglages

Définition : AWS CLI (Command Line Interface)

Programme en ligne de commande, publié par AWS, qui transforme une commande en appel signé à l'API d'un service, puis affiche la réponse.

Pourquoi l'utiliser

- **Reproductible** : une commande se copie, se rejoue, se versionne.
- **Automatisable** : scripts, tâches planifiées, chaînes d'intégration continue.
- **Complète** : toute l'API est accessible, y compris ce que la console ne montre pas encore.
- **Rapide** : lister 200 ressources ou en modifier 30 en une ligne.

Ce qu'il faut savoir

- Version actuelle : **la version 2**. La version 1 n'offre ni `aws login`, ni Identity Center, ni `aws logs tail`.
- Elle fonctionne sous macOS, Linux et Windows, et dans CloudShell où elle est préinstallée.
- Les SDK (Python, Java, JavaScript...) appellent les mêmes API et lisent les mêmes fichiers de configuration.

`txt` Forme générale d'une commande

```
aws <service> <opération> [paramètres de l'opération] [options générales]
aws ec2 describe-instances --instance-ids i-0abc123def4567890 --region eu-west-3 --output table
```

Correspondance avec IAM. L'opération `describe-instances` du service `ec2` est l'action `ec2:DescribeInstances` des politiques. Ce que la CLI peut faire, c'est ce que vos politiques autorisent.

\$ macOS et Linux

```
# macOS : installeur officiel
curl -s0 https://awscli.amazonaws.com/AWSCLIV2.pkg
sudo installer -pkg AWSCLIV2.pkg -target /

# Linux x86_64 (et WSL2)
curl -s https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip \
  -o awscliv2.zip
unzip -q awscliv2.zip
sudo ./aws/install

# Mise à jour sous Linux : même commande, avec --update
sudo ./aws/install --update

# Vérification, sur tous les systèmes
aws --version
which aws
```

PS Windows (PowerShell)

```
# Installation ou mise à jour
msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi

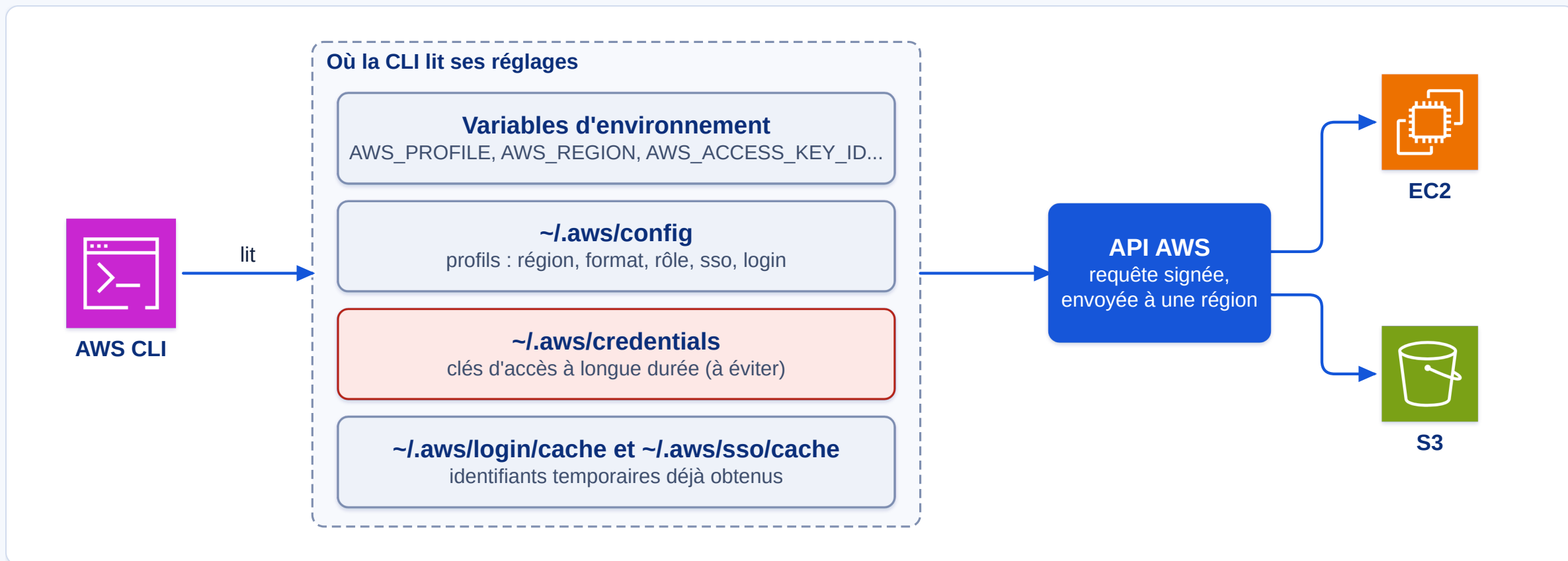
# Rouvrir PowerShell, puis vérifier
aws --version
```

Points de contrôle

- `aws --version` doit afficher `aws-cli/2.x`, en version 2.32 ou plus récente pour `aws login`.
- Si la version 1 répond, une ancienne installation (souvent par `pip`) passe devant : la désinstaller.
- Une commande inconnue, du type « invalid choice », signale presque toujours une version trop ancienne.

Sans installation. CloudShell fournit une CLI à jour dans le navigateur : utile en dépannage ou sur un poste verrouillé.

La CLI Schéma : où la CLI trouve ses réglages



Trois informations nécessaires

Pour chaque commande, la CLI doit savoir **qui** signe la requête (identifiants), **où** l'envoyer (région) et **comment** afficher la réponse (format).

Deux fichiers

`config` décrit les profils et ne contient aucun secret. `credentials` contient des clés permanentes : avec `aws login`, il reste vide.

Emplacement

Le dossier `.aws` du dossier personnel : `~/.aws` sous macOS et Linux, `%USERPROFILE%\ .aws` sous Windows.

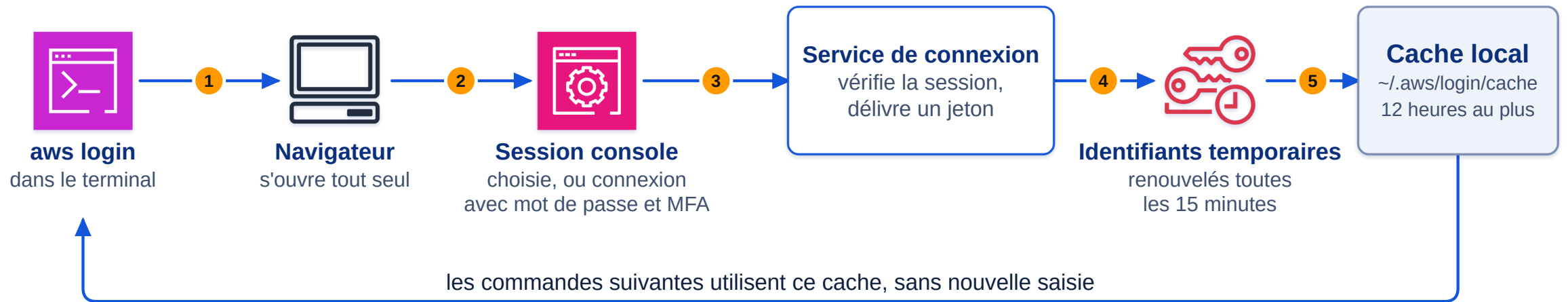
La CLI Les façons de s'authentifier dans la CLI

Méthode	Commande	Identifiants obtenus	Pour qui
Connexion console	<code>aws login</code>	Temporaires, renouvelés automatiquement.	Toute identité capable de se connecter à la console : utilisateur IAM, racine, fédéré.
IAM Identity Center	<code>aws configure sso</code> puis <code>aws sso login</code>	Temporaires, pour le compte et le rôle choisis.	Entreprises équipées d'Identity Center.
Rôle à endosser	Profil avec <code>role_arn</code>	Temporaires, ceux du rôle.	En complément d'une des deux précédentes.
Clés d'accès	<code>aws configure</code>	Permanents, écrits en clair dans <code>credentials</code> .	À défaut d'autre possibilité.
Rôle de la machine	Aucune	Temporaires, fournis par EC2, ECS ou Lambda.	Code qui tourne sur AWS.

Ordre de préférence. Identity Center ou `aws login` pour les personnes, rôle de service pour le code. Les clés d'accès en dernier recours, parce qu'elles ne s'éteignent jamais seules.

Dans la formation. `aws login` avec son utilisateur, puis un profil qui endosse `RoleApprenants`.

Schéma : le fonctionnement de aws login



1. La commande démarre un petit serveur local et ouvre le navigateur sur la page de connexion d'AWS.
2. Si une session console est déjà ouverte, on la choisit ; sinon on se connecte, second facteur compris.
3. Le navigateur renvoie à la CLI une preuve de cette connexion.
4. La CLI l'échange contre des identifiants temporaires correspondant à l'identité de la console.
5. Ils sont rangés dans un cache local, et renouvelés sans intervention tant que la session reste valable.

Ce que cela change. Aucune clé d'accès n'est créée ni copiée : le poste ne contient que des identifiants à durée courte. Disponible à partir de la version 2.32 de la CLI.

\$ Prérequis : AWS CLI 2.32 ou plus, un utilisateur capable de se connecter à la console

Connexion : le navigateur s'ouvre

```
aws login
```

Quelle identité la CLI utilise-t-elle ?

```
aws sts get-caller-identity
```

Région par défaut du profil

```
aws configure set region eu-west-3
```

Vue d'ensemble des réglages en cours

```
aws configure list
```

Sans navigateur sur la machine (serveur distant) :

la CLI affiche une adresse à ouvrir ailleurs

```
aws login --remote
```

Avec un profil nommé

```
aws login --profile formation
```

Fin de session : efface les identifiants du cache

```
aws logout
```

`{}` Réponse de `aws sts get-caller-identity`

```
{
  "UserId": "AIDAEXAMPLEUSERID123",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/user7"
}
```

Lecture

- `Account` : le compte dans lequel on agit.
- `Arn` : le principal réel. C'est lui qui figure dans les messages `AccessDenied` et dans CloudTrail.
- Cette commande ne demande aucun droit : c'est le premier réflexe en cas de doute.

Droit nécessaire. Un utilisateur IAM aux droits restreints doit être autorisé à utiliser cette connexion : politique gérée `SignInLocalDevelopmentAccess`. Sans elle, la connexion aboutit dans le navigateur mais la CLI est refusée.

Définition : profil

Ensemble nommé de réglages de la CLI : une identité, une région, un format de sortie. On en définit autant qu'on a de comptes ou de rôles à utiliser.

```
ini ~/.aws/config
```

```
[default]
region = eu-west-3
output = json

[profile formation]
region = eu-west-3
output = table

[profile apprenant]
role_arn = arn:aws:iam::123456789012:role/RoleApprenants
source_profile = default
region = eu-west-3

[profile prod-lecture]
sso_session = entreprise
sso_account_id = 999988887777
sso_role_name = ReadOnly
region = eu-west-3
```

```
$ Choisir un profil
```

```
# Pour une seule commande
aws s3 ls --profile apprenant

# Pour tout le terminal (bash, zsh)
export AWS_PROFILE=apprenant

# Lister les profils connus
aws configure list-profiles

# Lire ou écrire une valeur d'un profil
aws configure get region --profile apprenant
aws configure set output table --profile apprenant
```

Le profil default est utilisé quand rien n'est précisé. Tous les autres s'écrivent `[profile nom]` dans `config`.

Piège classique. Lancer une commande destructrice avec le mauvais profil. Bon réflexe : `aws sts get-caller-identity` avant toute suppression, et un profil par défaut sans droits d'écriture en production.

La CLI La région dans la CLI

Pourquoi elle compte

- Chaque service régional a une adresse par région : `ec2.eu-west-3.amazonaws.com`. La CLI doit savoir laquelle appeler.
- Sans région : « You must specify a region ».
- Avec la mauvaise région : la commande réussit, mais renvoie une liste vide, ou crée la ressource ailleurs.

#	D'où la CLI tire la région
1	L'option <code>--region</code> de la commande.
2	La variable <code>AWS_REGION</code> .
3	La variable <code>AWS_DEFAULT_REGION</code> .
4	La ligne <code>region</code> du profil utilisé.

\$ Régler et vérifier la région

```
# Dans le profil
aws configure set region eu-west-3

# Pour le terminal en cours
export AWS_REGION=eu-west-3

# Pour une seule commande
aws ec2 describe-instances --region us-east-1

# Quelle région sera utilisée, et d'où vient-elle ?
aws configure list

# Parcourir toutes les régions activées
for r in $(aws ec2 describe-regions \
  --query 'Regions[].RegionName' --output text); do
  echo "$r : $(aws ec2 describe-instances --region $r \
    --query 'length(Reservations[].Instances[])')"
```

Services mondiaux. Les commandes `aws iam` ne dépendent pas de la région. `aws s3 ls` liste les buckets de toutes les régions.

La CLI

Le format de sortie

Format	Caractéristique	Usage
json	Structure complète, format par défaut.	Scripts, traitement par <code>jq</code> .
yaml	Même contenu, plus lisible.	Lecture à l'écran.
table	Tableau encadré avec titres.	Démonstration, vérification rapide.
text	Valeurs séparées par des tabulations, sans titres.	Variables de shell, boucles.

\$ Régler le format

```
# Pour une commande
aws ec2 describe-instances --output table --query \
'Reservations[].Instances[].[InstanceId,InstanceType,State.Name]'
```

```
# Par défaut, dans le profil
aws configure set output json
```

```
# Pour le terminal
export AWS_DEFAULT_OUTPUT=table
```

txt Trois champs par instance, en table

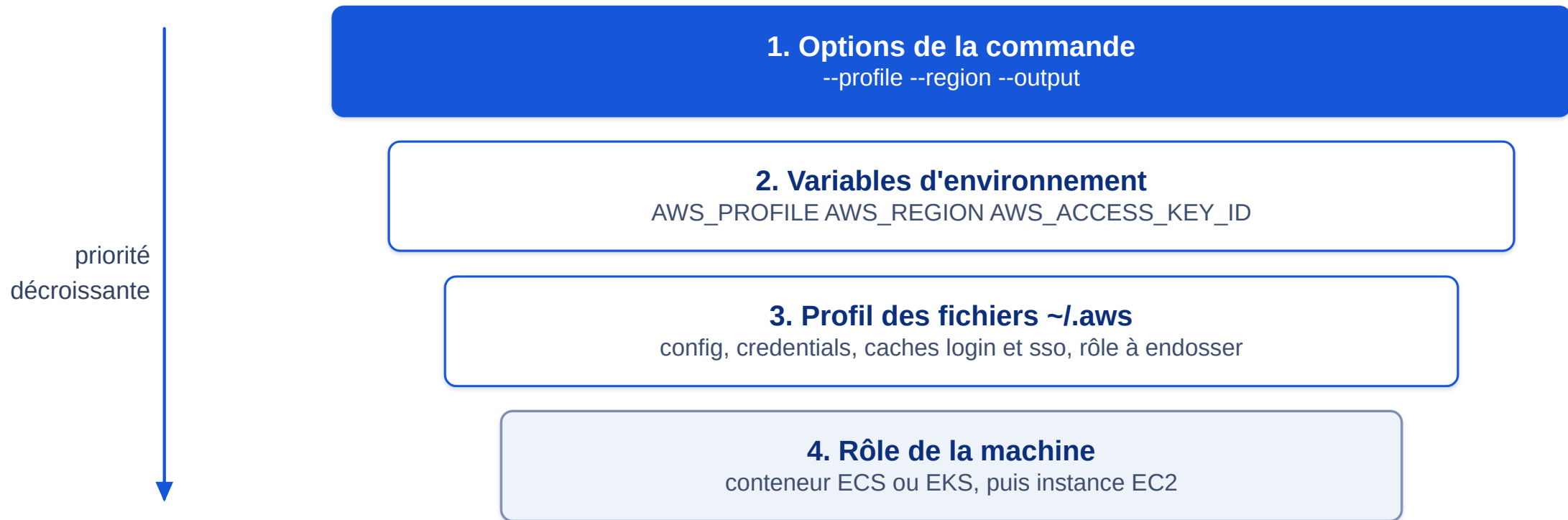
```
-----
|                               DescribeInstances                               |
+-----+-----+-----+-----+
| i-0abc123def4567890 | t3.micro | running |
| i-0def456abc7890123 | t3.small | stopped  |
+-----+-----+-----+-----+
```

txt Les mêmes, en text

```
i-0abc123def4567890    t3.micro    running
i-0def456abc7890123    t3.small    stopped
```

Le pager. En version 2, les réponses longues s'ouvrent dans un afficheur page par page (touche `q` pour en sortir). Pour l'éviter : `export AWS_PAGER=""` ou l'option `--no-cli-pager`.

Schéma : l'ordre de priorité des réglages



Règle. Le réglage le plus proche de la commande l'emporte. Une option gagne sur une variable d'environnement, qui gagne sur le fichier de configuration.

Source de confusion. Une variable `AWS_PROFILE` ou `AWS_ACCESS_KEY_ID` restée dans le terminal passe avant le profil du fichier. Quand une commande n'agit pas avec l'identité attendue : `aws configure list` indique d'où vient chaque valeur.

Variable	Rôle
AWS_PROFILE	Profil à utiliser.
AWS_REGION, AWS_DEFAULT_REGION	Région.
AWS_DEFAULT_OUTPUT	Format de sortie.
AWS_PAGER	Afficheur page par page ; vide pour le désactiver.
AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN	Identifiants passés directement, par exemple par une chaîne d'intégration continue.
AWS_CONFIG_FILE, AWS_SHARED_CREDENTIALS_FILE	Emplacement des fichiers.

Option	Effet
--profile, --region, --output	Remplacent le réglage par défaut pour cette commande.
--query	Extrait une partie de la réponse.
--no-cli-pager	Affiche tout, sans pagination d'écran.
--debug	Détaille la requête envoyée et la réponse brute.
--dry-run	Pour EC2 : vérifie les droits sans exécuter.
--cli-input-json	Lit les paramètres dans un fichier.
help	Après un service ou une opération : la documentation.

Définition : **waiter**

Sous-commande `aws <service> wait ...` qui bloque jusqu'à ce qu'une ressource atteigne un état donné : instance démarrée, base disponible. Indispensable dans un script.

Pagination des résultats. Une API renvoie ses résultats par pages. La CLI les enchaîne toute seule ; `--max-items` limite le total, `--page-size` règle la taille des pages demandées.

La CLI et les rôles, partie 2

Filtrer les résultats

Une réponse d'API fait vite plusieurs centaines de lignes : apprendre à n'en garder que ce qui sert.

- Le problème : la réponse brute
- Filtrer côté serveur, filtrer côté client
- JMESPath : sélectionner
- Projeter, filtrer, trier
- Sortie pour les scripts
- Recettes et pièges

Le problème : une réponse brute trop riche

`{}` aws ec2 describe-instances (très abrégé : une instance, une centaine de lignes en réalité)

```
{
  "Reservations": [
    {
      "ReservationId": "r-0a1b2c3d4e5f67890",
      "Instances": [
        {
          "InstanceId": "i-0abc123def4567890",
          "InstanceType": "t3.micro",
          "State": { "Code": 16, "Name": "running" },
          "PrivateIpAddress": "10.0.1.25",
          "Placement": { "AvailabilityZone": "eu-west-3a" },
          "Tags": [
            { "Key": "Name", "Value": "user7-web-01" },
            { "Key": "Owner", "Value": "user7" }
          ]
        }
      ]
    }
  ]
}
```

Lire la structure

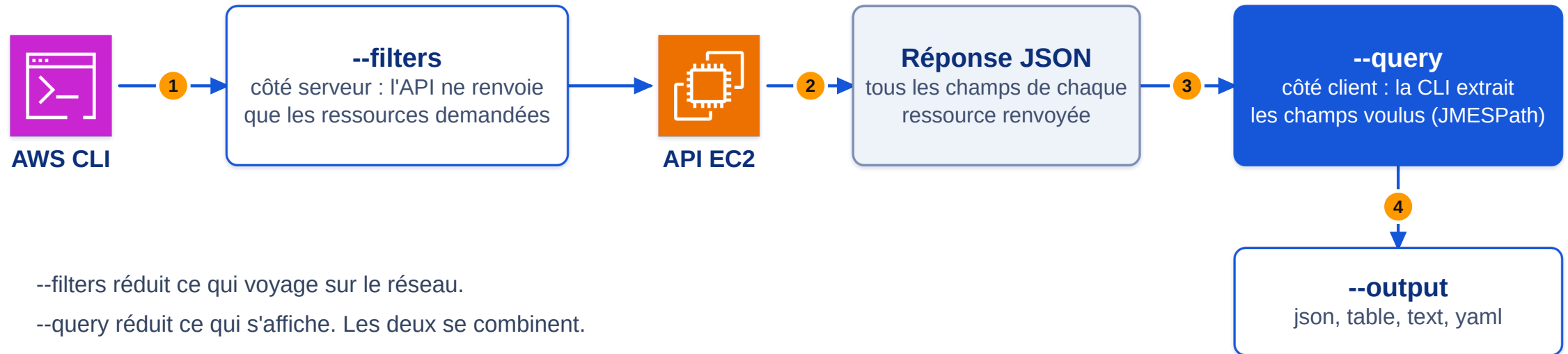
- `{ }` : un **objet**, des paires clé-valeur.
- `[ ]` : une **liste** d'éléments.
- Ici : un objet contenant la liste `Reservations` ; chaque réservation contient la liste `Instances` ; chaque instance contient des champs, des objets (`State`) et une liste (`Tags`).

Ce que l'on veut le plus souvent

- Trois ou quatre champs par ressource.
- Seulement certaines ressources : celles qui tournent, celles d'un propriétaire.
- Une valeur seule, pour la donner à la commande suivante.

Deux outils répondent à ce besoin : `--filters` et `--query`. Ils n'agissent pas au même endroit.

Schéma : filtrer côté serveur ou côté client



Définition : filtrage côté serveur

Sélection faite par le service AWS avant d'envoyer la réponse, grâce à des paramètres de l'opération comme `--filters`. Disponible pour certaines opérations seulement.

Définition : filtrage côté client

Tri et extraction faits par la CLI sur la réponse déjà reçue, avec l'option `-query`. Disponible pour toutes les commandes.

Définition : JMESPath

Langage de requête pour documents JSON, utilisé par l'option `--query`. Une expression décrit le chemin à suivre dans la réponse et ce qu'il faut en garder.

Expression	Signification	Résultat sur l'exemple
<code>Reservations</code>	La valeur d'une clé.	La liste des réservations.
<code>Reservations[0]</code>	Un élément par sa position, à partir de 0.	La première réservation.
<code>Reservations[-1]</code>	Le dernier élément.	La dernière réservation.
<code>Reservations[0].Instances[0].InstanceId</code>	Un chemin : le point descend d'un niveau.	<code>"i-0abc123def4567890"</code>
<code>Reservations[].Instances[]</code>	<code>[]</code> : tous les éléments, mis à plat.	Toutes les instances, en une seule liste.
<code>Reservations[].Instances[].InstanceId</code>	Un champ de chaque élément.	<code>["i-0abc...", "i-0def..."]</code>
<code>Reservations[].Instances[].State.Name</code>	Un champ d'un objet imbriqué.	<code>["running", "stopped"]</code>
<code>Reservations[0:2]</code>	Une tranche : positions 0 et 1.	Les deux premières réservations.

Le réflexe pour EC2. Presque toutes les requêtes sur les instances commencent par `Reservations[].Instances[]`, qui traverse les deux listes imbriquées et rend une liste simple d'instances.

Choisir les champs : listes et objets

\$ Plusieurs champs sous forme de liste

```
aws ec2 describe-instances --query \
'Reservations[].Instances[].[InstanceId, InstanceType, State.Name]'
```

{ } Résultat

```
[
  [ "i-0abc123def4567890", "t3.micro", "running" ],
  [ "i-0def456abc7890123", "t3.small", "stopped" ]
]
```

Les crochets `[a, b, c]` produisent une ligne par ressource. Avec `--output table`, on obtient un tableau sans titres de colonnes.

\$ Plusieurs champs sous forme d'objet, avec des noms choisis

```
aws ec2 describe-instances --query \
'Reservations[].Instances[].{id:InstanceId,type:InstanceType,etat:State.Name}'
```

{ } Résultat

```
[
  { "id": "i-0abc123def4567890", "type": "t3.micro",
    "etat": "running" },
  { "id": "i-0def456abc7890123", "type": "t3.small",
    "etat": "stopped" }
]
```

Les accolades `{nom: champ}` nomment chaque valeur. Avec `--output table`, les noms deviennent les titres des colonnes.

Définition : projection

Application d'une même expression à chaque élément d'une liste, pour en produire une nouvelle : c'est ce que font `[]` et `[*]`.

Filtrer les résultats

Filtrer : ne garder que certains éléments

Expression	Garde
<code>Reservations[].Instances[?State.Name== 'running']</code>	Les instances en cours d'exécution.
<code>Reservations[].Instances[?InstanceType!= 't3.micro']</code>	Celles d'un autre type.
<code>Volumes[?Size > `100`]</code>	Les volumes de plus de 100 Go.
<code>Volumes[?State== 'available' && Size >= `50`]</code>	Volumes détachés d'au moins 50 Go.
<code>Users[?starts_with(UserName, 'user')]</code>	Les utilisateurs dont le nom commence par user.
<code>Reservations[].Instances[?contains(InstanceType, 'large')]</code>	Les instances de taille large.
<code>Reservations[].Instances[?Placement.AvailabilityZone== 'eu-west-3a']</code>	Celles de la zone a.
<code>Reservations[].Instances[?!PublicIpAddress]</code>	Celles qui n'ont pas d'adresse publique.

Syntaxe du filtre

`[?condition]` se place après une liste. Opérateurs : `==`, `!=`, `<`, `<=`, `>`, `>=`, `&&` (et), `||` (ou), `!` (non).

Valeurs littérales

Texte : entre apostrophes, `'running'`. Nombres et booléens : entre accents graves, ``100``, ``true``. Les oublier donne un résultat vide, sans message d'erreur.

Fonction	Rôle
<code>length(liste)</code>	Nombre d'éléments.
<code>sort_by(liste, &champ)</code>	Trier selon un champ.
<code>reverse(liste)</code>	Inverser l'ordre.
<code>max_by(liste, &champ)</code> , <code>min_by</code>	L'élément le plus grand, le plus petit.
<code>sum(liste)</code> , <code>avg(liste)</code>	Somme, moyenne de nombres.
<code>contains(a, b)</code> , <code>starts_with</code> , <code>ends_with</code>	Tests sur du texte ou une liste.
<code>join(', ', liste)</code>	Réunir des textes.
<code>keys(objet)</code>	Les noms des champs.

\$ Exemples

```
# Combien d'instances ?
aws ec2 describe-instances \
  --query 'length(Reservations[].Instances[])'

# Les 3 buckets les plus récents
aws s3api list-buckets --query \
  'reverse(sort_by(Buckets, &CreationDate))[:3].Name'

# Taille totale des volumes, en Go
aws ec2 describe-volumes --query 'sum(Volumes[].Size)'

# La première instance en cours d'exécution
aws ec2 describe-instances --query \
  "Reservations[].Instances[?State.Name=='running'][] | [0].InstanceId"
```

Définition : tube (pipe)

Le signe `|` arrête la projection en cours et passe son résultat entier à l'expression suivante. Nécessaire pour prendre « le premier de la liste obtenue ».

\$ Récupérer une valeur dans une variable

Numéro du compte

```
ACCOUNT_ID=$(aws sts get-caller-identity \
  --query Account --output text)
```

Identifiant du VPC par défaut

```
VPC_ID=$(aws ec2 describe-vpcs \
  --filters Name=isDefault,Values=true \
  --query 'Vpcs[0].VpcId' --output text)
```

Identifiant d'une instance d'après son nom

```
ID=$(aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=user7-web-01" \
  --query 'Reservations[0].Instances[0].InstanceId' \
  --output text)
```

```
echo "$ACCOUNT_ID $VPC_ID $ID"
```

Boucler sur une liste de valeurs

```
for b in $(aws s3api list-buckets \
  --query 'Buckets[].Name' --output text); do
  echo "bucket : $b"
done
```

Pourquoi text

- En `json`, la valeur arrive entre guillemets : `"vpc-0abc..."`. Inutilisable telle quelle dans la commande suivante.
- En `text`, elle arrive nue : `vpc-0abc...`.
- Une liste devient des valeurs séparées par des tabulations, que le shell sait parcourir.

Valeur absente. Si rien ne correspond, `--output text` renvoie le mot `None`. Un script doit le tester avant de continuer, sinon la commande suivante échoue de façon obscure.

Équivalent PowerShell. `$VPC_ID = aws ec2 describe-vpcs --query 'Vpcs[0].VpcId' --output text`. Les variables d'environnement s'y écrivent `$env:AWS_REGION`.

\$ Prérequis : une identité connectée et une région définie

```
# Instances : nom, identifiant, type, état, zone
aws ec2 describe-instances --output table --query \
  "Reservations[].Instances[].{nom:Tags[?Key=='Name']|[0].Value,id:InstanceId,type:InstanceType,etat:State.Name,zone:Placement.AvailabilityZone}"

# Volumes détachés, donc facturés pour rien
aws ec2 describe-volumes --filters Name=status,Values=available \
  --query 'Volumes[][VolumeId,Size,CreateTime]' --output table

# Utilisateurs IAM et date de création, du plus récent au plus ancien
aws iam list-users --query 'reverse(sort_by(Users, &CreateDate))[][UserName,CreateDate]' --output table

# Politiques attachées à un rôle
aws iam list-attached-role-policies --role-name RoleApprenants --query 'AttachedPolicies[].PolicyName'

# Security groups qui ouvrent le port 22 à tout Internet
aws ec2 describe-security-groups --output table \
  --filters Name=ip-permission.from-port,Values=22 Name=ip-permission.cidr,Values=0.0.0.0/0 \
  --query 'SecurityGroups[][GroupId,GroupName]'

# Bases RDS : identifiant, moteur, classe, état
aws rds describe-db-instances --output table \
  --query 'DBInstances[][DBInstanceIdentifier,Engine,DBInstanceClass,DBInstanceStatus]'
```

Le nom d'une ressource EC2 est une étiquette parmi d'autres. D'où l'expression `Tags[?Key=='Name']|[0].Value` : filtrer la liste des étiquettes sur la clé Name, prendre le premier résultat, lire sa valeur.

--query : pièges et bonnes pratiques

Piège	Ce qui se passe	Parade
Guillemets du shell	L'expression contient des apostrophes : elle doit alors être entourée de guillemets doubles, ou l'inverse.	Entourer de guillemets doubles dès qu'il y a un filtre avec du texte.
Accents graves en bash	Entre guillemets doubles, le shell les prend pour une commande à exécuter.	Entourer d'apostrophes quand l'expression contient des accents graves.
Casse des noms de champs	<code>instanceId</code> au lieu de <code>InstanceId</code> : résultat <code>null</code> , sans erreur.	Copier les noms depuis la réponse JSON brute.
Nombre comparé comme un texte	Résultat vide.	Accents graves autour des nombres.
Oubli du tube avant <code>[0]</code>	On obtient le premier élément de chaque sous-liste, pas le premier résultat.	<code>... [0]</code>
Filtrer avec --query ce que l'API sait filtrer	Toute la liste est transférée avant d'être réduite.	Utiliser <code>--filters</code> d'abord quand il existe.
Expression illisible	Personne ne sait la corriger.	La construire étape par étape, en vérifiant chaque résultat.

Méthode de construction. Afficher d'abord la réponse brute. Ajouter le chemin jusqu'à la liste voulue. Puis le filtre. Puis le choix des champs. Puis le format de sortie.

Et jq ? L'outil `jq` fait la même chose à partir du JSON, avec un autre langage. `--query` a l'avantage d'être intégré et identique sur tous les systèmes.

La CLI et les rôles, partie 3

Rôles et identifiants temporaires

Ce qui se passe quand on endosse un rôle : la trust policy, le service STS, la session.

- STS et les identifiants temporaires
- AssumeRole pas à pas
- La trust policy
- Endosser un rôle avec la CLI
- Sessions : durée, nom, traçabilité
- Accès entre comptes et fédération

Définition : AWS STS (Security Token Service)

Service qui délivre des identifiants à durée limitée. C'est lui que l'on appelle, directement ou non, chaque fois que l'on endosse un rôle.

Définition : identifiants temporaires

Ensemble formé d'un identifiant de clé (`ASIA...`), d'un secret et d'un jeton de session, assorti d'une date d'expiration après laquelle il ne vaut plus rien.

Ce qui les distingue d'une clé permanente

- **Ils expirent seuls** : pas de rotation à organiser, pas de clé oubliée.
- **Ils ne sont pas stockés chez l'utilisateur** : générés à la demande, renouvelés par la CLI ou le SDK.
- **Ils portent un jeton de session**, obligatoire dans chaque requête.
- **Ils ne peuvent pas être révoqués un par un**, mais on peut invalider toutes les sessions d'un rôle ouvertes avant une date.

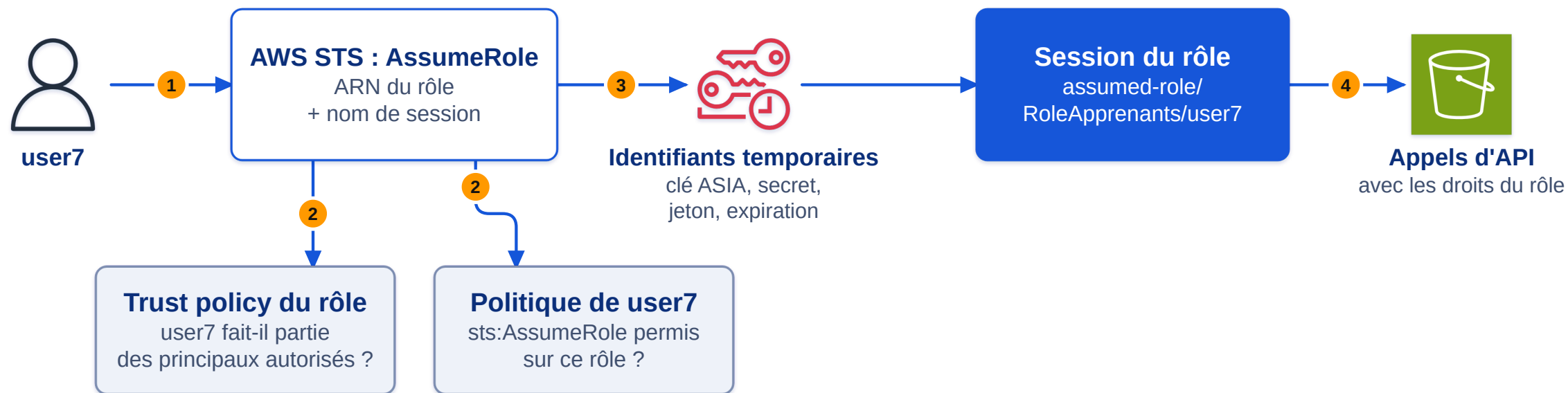
Où on les rencontre

- Dans la console, après connexion.
- Dans la CLI, après `aws login` ou une connexion Identity Center.
- À chaque prise de rôle, par une personne ou par un service.
- Dans une instance EC2, une fonction Lambda, une tâche ECS.

Autrement dit : presque partout. La clé d'accès permanente d'un utilisateur IAM est l'exception.

STS est gratuit, et joignable dans chaque région. Ses appels sont journalisés dans CloudTrail comme les autres.

Schéma : AssumeRole pas à pas



1. user7 appelle `sts:AssumeRole` en indiquant l'ARN du rôle et un nom de session.

2. STS vérifie la trust policy du rôle, et les droits de user7 sur l'action `sts:AssumeRole`.

3. STS renvoie des identifiants temporaires : clé, secret, jeton de session, date d'expiration.

4. Les appels signés avec ces identifiants sont évalués avec les politiques du rôle, et plus avec celles de user7.

 Trust policy de RoleApprenants

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:root"
      },
      "Action": "sts:AssumeRole",
      "Condition": {
        "Bool": {
          "aws:MultiFactorAuthPresent": "true"
        }
      }
    }
  ]
}
```

Élément	Lecture
Principal	Le compte 123456789012 dans son ensemble : il revient ensuite à ce compte de dire lesquelles de ses identités peuvent endosser le rôle.
Action	<code>sts:AssumeRole</code> : la seule chose qu'une trust policy autorise, c'est d'endosser.
Condition	Facultative. Ici : seulement si un second facteur a été présenté.

C'est une politique de ressource, dont la ressource est le rôle lui-même. Elle se lit avec la même grille : effet, principal, action, condition.

Ne jamais écrire `"Principal": "*" ou {"AWS": "*"}` dans une trust policy : n'importe quel compte AWS pourrait endosser le rôle.

Rôles et STS

Les deux autorisations nécessaires pour endosser un rôle

Situation	Trust policy du rôle	Politique d'identité de celui qui endosse
Même compte, la trust policy cite précisément l'utilisateur	Autorise <code>user/user7</code> .	Rien à ajouter : la trust policy suffit.
Même compte, la trust policy cite le compte (<code>:root</code>)	Délègue la décision au compte.	Doit autoriser <code>sts:AssumeRole</code> sur l'ARN du rôle.
Deux comptes différents	Autorise le compte ou l'identité externe.	Doit autoriser <code>sts:AssumeRole</code> sur l'ARN du rôle. Les deux sont obligatoires.

`{ }` Politique d'identité attachée à user7

```
{ "Version": "2012-10-17",
  "Statement": [ {
    "Effect": "Allow",
    "Action": "sts:AssumeRole",
    "Resource":
      "arn:aws:iam::123456789012:role/RoleApprenants"
  } ] }
```

C'est le montage de la formation. La trust policy de `RoleApprenants` cite le compte ; chaque utilisateur reçoit en plus cette politique, qui l'autorise à endosser ce rôle et aucun autre.

Erreur fréquente. « is not authorized to perform: sts:AssumeRole » : l'un des deux côtés manque. Vérifier l'ARN du rôle dans la politique de l'utilisateur, puis le principal de la trust policy.

Opération	Qui l'appelle	Ce qu'elle renvoie
GetCallerIdentity	N'importe qui, sans droit particulier.	Le compte et l'ARN de l'identité en cours.
AssumeRole	Un utilisateur, un rôle, un service AWS.	Les identifiants temporaires d'un rôle.
AssumeRoleWithSAML	Une personne authentifiée par un annuaire d'entreprise.	Les identifiants d'un rôle, sans utilisateur IAM.
AssumeRoleWithWebIdentity	Une identité OIDC : chaîne d'intégration continue, pod Kubernetes, application mobile.	Les identifiants d'un rôle.
GetSessionToken	Un utilisateur IAM qui présente son second facteur.	Des identifiants temporaires portant ses propres droits, marqués MFA.

Durée d'une session de rôle	Valeur
Par défaut	1 heure
Maximum réglable sur le rôle	De 1 à 12 heures
Minimum demandable	15 minutes
Rôle endossé depuis un autre rôle	1 heure au plus

Définition : enchaînement de rôles (role chaining)

Fait d'endosser un rôle à partir d'une session qui provient déjà d'un rôle. La session obtenue est limitée à une heure.

La durée maximale se règle rôle par rôle (`MaxSessionDuration`). Plus la session est courte, plus une fuite d'identifiants est vite sans effet.

ini Méthode recommandée : un profil dans ~/.aws/config

```
[default]
region = eu-west-3

[profile apprenant]
role_arn = arn:aws:iam::123456789012:role/RoleApprenants
source_profile = default
role_session_name = user7
region = eu-west-3
```

\$ Utilisation

```
aws login                # identité de départ
aws sts get-caller-identity --profile apprenant
aws s3 ls --profile apprenant
export AWS_PROFILE=apprenant # pour tout le terminal
```

La CLI appelle STS toute seule, garde les identifiants en cache et les renouvelle à l'expiration.

\$ Méthode manuelle : pour comprendre ce que fait le profil

```
# 1. Demander les identifiants du rôle
aws sts assume-role \
  --role-arn arn:aws:iam::123456789012:role/RoleApprenants \
  --role-session-name user7-demo \
  --duration-seconds 3600 > creds.json

# 2. Regarder la réponse : quatre valeurs
#   AccessKeyId, SecretAccessKey, SessionToken, Expiration
jq '.Credentials | keys' creds.json

# 3. Les placer dans des variables d'environnement
C=.Credentials
export AWS_ACCESS_KEY_ID=$(jq -r $C.AccessKeyId creds.json)
export AWS_SECRET_ACCESS_KEY=$(jq -r $C.SecretAccessKey creds.json)
export AWS_SESSION_TOKEN=$(jq -r $C.SessionToken creds.json)

# 4. Vérifier : l'ARN est maintenant celui d'une session de rôle
aws sts get-caller-identity

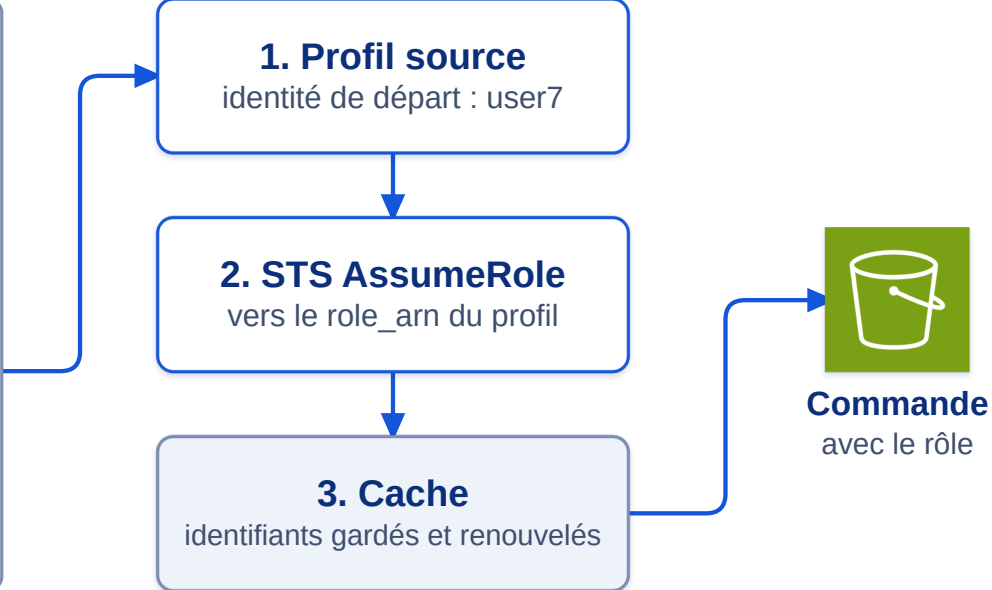
# 5. Revenir à l'identité de départ, puis effacer le fichier
unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN
rm creds.json
```

Schéma : ce que fait un profil avec role_arn

```
~/aws/config, utilisé avec --profile apprenant

[default]
region = eu-west-3

[profile apprenant]
role_arn = arn:aws:iam::123456789012:role/RoleApprenants
source_profile = default
region = eu-west-3
```



Sur une ressource AWS, on remplace `source_profile` par `credential_source = Ec2InstanceMetadata` ou `EcsContainer` : le rôle de la machine sert alors d'identité de départ pour endosser un autre rôle.

```
{ aws sts get-caller-identity, une fois le rôle endossé
```

```
{
  "UserId": "AROEXAMPLEID456:user7",
  "Account": "123456789012",
  "Arn": "arn:aws:sts::123456789012:assumed-role/RoleApprenants/user7"
}
```

Partie de l'ARN	Signification
<code>sts</code>	C'est une session délivrée par STS, pas un objet d'IAM.
<code>assumed-role</code>	Elle provient d'un rôle endossé.
<code>RoleApprenants</code>	Le rôle dont elle porte les droits.
<code>user7</code>	Le nom de session : ce qui distingue deux personnes sous le même rôle.

Définition : nom de session

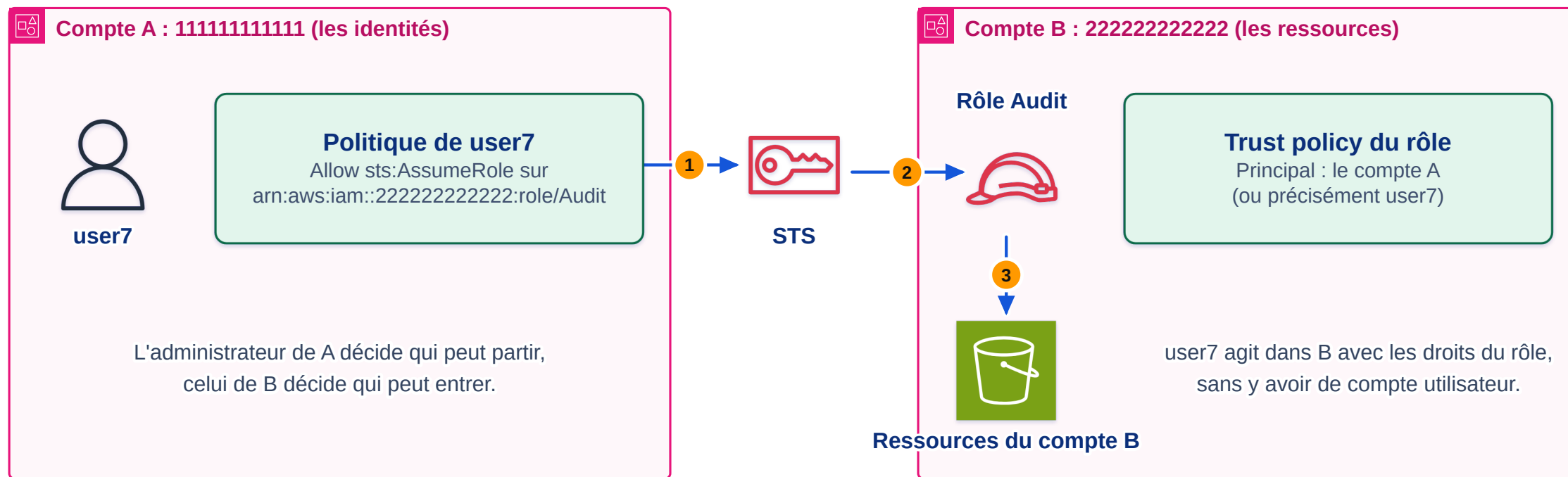
Libellé choisi au moment d'endosser un rôle, qui figure dans l'ARN de la session et dans chaque événement CloudTrail.

Pourquoi il compte

- Quinze apprenants partagent `RoleApprenants` : sans nom de session, impossible de savoir qui a supprimé quoi.
- Depuis la console, AWS met le nom de l'utilisateur comme nom de session.
- En CLI, c'est à vous de le fixer (`role_session_name`) ; une trust policy peut l'imposer avec la clé `sts:RoleSessionName`.

Préfixes d'identifiants. `AIDA` : utilisateur. `AROA` : rôle. `AKIA` : clé permanente. `ASIA` : clé temporaire. Utiles pour lire un journal.

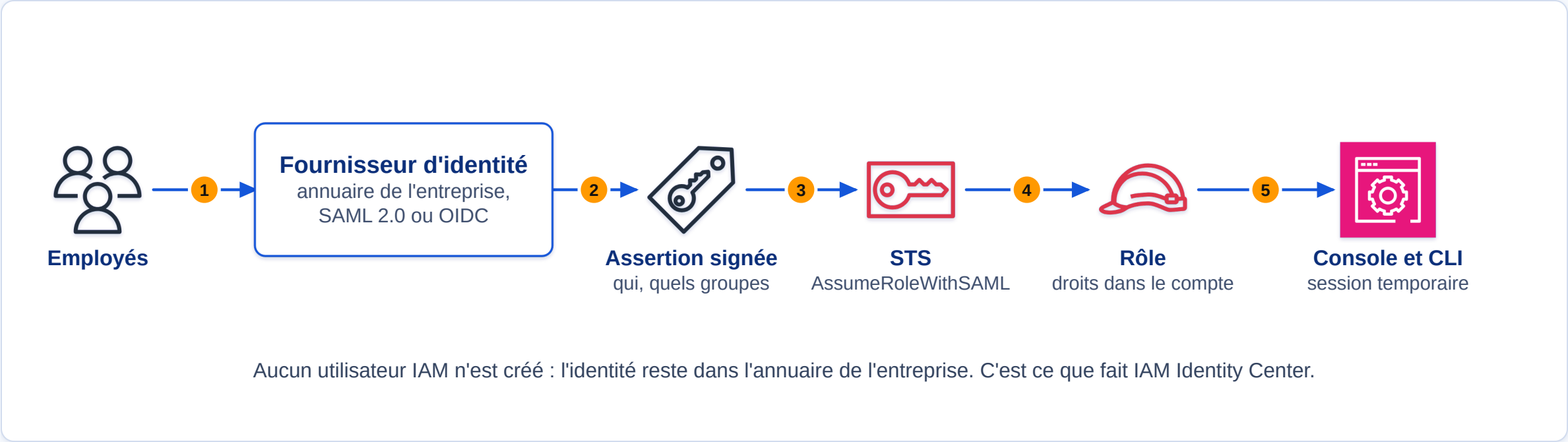
Schéma : l'accès entre comptes



1. user7, dans le compte A, appelle STS pour endosser le rôle Audit du compte B. Sa politique d'identité doit le lui permettre.
2. STS vérifie que la trust policy du rôle, dans le compte B, admet le compte A.
3. Avec les identifiants obtenus, user7 agit sur les ressources du compte B, dans la limite des droits du rôle.

Intérêt. Un seul endroit pour les identités, des rôles dans chaque compte : c'est le fondement des architectures à plusieurs comptes. Quand le compte A appartient à un tiers (prestataire, outil SaaS), on ajoute dans la trust policy la condition `sts:ExternalId`, un secret partagé qui empêche ce tiers d'être manipulé pour accéder à votre compte au nom d'un autre de ses clients.

Schéma : la fédération d'identités



Définition : fédération d'identités

Mécanisme par lequel AWS fait confiance à un fournisseur d'identité externe pour authentifier les utilisateurs, puis leur accorde un rôle, sans créer d'utilisateur IAM.

Cas	Fournisseur d'identité
Salariés	Annuaire de l'entreprise, par SAML ; en pratique via IAM Identity Center.
Intégration continue	GitHub Actions, GitLab CI, par OIDC : plus aucune clé AWS dans les variables du pipeline.
Pods Kubernetes	Le cluster EKS, par OIDC ou Pod Identity.
Utilisateurs d'une application	Amazon Cognito.

Pratique	Raison
Trust policy la plus étroite possible : un rôle ou un utilisateur précis plutôt que tout un compte	Réduit le nombre d'identités capables d'endosser le rôle.
Exiger un second facteur pour les rôles puissants	Des identifiants volés ne suffisent plus à élever ses droits.
Durée de session courte pour les rôles sensibles	Limite la fenêtre d'exploitation d'une fuite.
Imposer un nom de session identifiant la personne	Garantit la traçabilité dans CloudTrail.
Un rôle par fonction, pas un rôle fourre-tout	Moindre privilège, et lecture facile des droits.
Condition <code>sts:ExternalId</code> pour tout tiers	Protège contre l'usurpation par l'intermédiaire du tiers.
Surveiller les appels AssumeRole dans CloudTrail	Qui endosse quoi, depuis où, à quelle fréquence.

Révoquer en urgence. Dans la console, onglet **Revoke sessions** du rôle : AWS ajoute une politique qui refuse tout aux sessions ouvertes avant l'instant présent. Les nouvelles sessions ne sont pas touchées.

Un droit temporairement élevé. Travailler au quotidien avec des droits limités, et n'endosser un rôle d'administration que le temps d'une opération : c'est l'application du moindre privilège à la durée.

La CLI et les rôles, partie 4

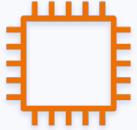
Les rôles de service

Comment une instance EC2, une fonction Lambda ou une tâche ECS appellent AWS sans qu'aucune clé ne soit écrite nulle part.

- Le problème des clés dans le code
- Rôle de service et rôle lié à un service
- EC2 : instance profile et métadonnées
- Lambda : rôle d'exécution
- ECS : task role et execution role
- iam:PassRole et chaîne des identifiants

Schéma : des clés dans le code, ou un rôle

À proscrire : des clés dans le code ou la configuration



Instance

```
aws_access_key_id = AKIA...EXAMPLE  
aws_secret_access_key = wJal...KEY
```

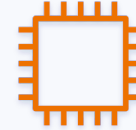
Copiées dans Git, les sauvegardes, les images de machine.

Jamais renouvelées : valables des années.

Les mêmes droits partout où le fichier a traîné.

Première cause de compromission de comptes AWS.

À faire : un rôle attaché à la ressource



Instance

attaché



Rôle

fournit



Identifiants temporaires

Rien à stocker, rien à distribuer.

Renouvelés automatiquement, toutes les quelques heures.

Des droits propres à chaque service.

Aucun changement dans le code : le SDK les trouve seul.

Définition : rôle de service

Rôle IAM qu'un service AWS endosse pour agir en votre nom. Sa trust policy désigne le service comme principal, par exemple

```
ec2.amazonaws.com.
```

Le principe. Le code ne contient aucun identifiant. Il demande à AWS « quels sont mes droits ? », et la plateforme sur laquelle il tourne lui remet les identifiants temporaires du rôle qu'on lui a attaché.

Rôles de service

Rôle de service et rôle lié à un service

	Rôle de service	Rôle lié à un service
Créé par	Vous.	Le service lui-même, à la première utilisation.
Droits	Choisis par vous.	Fixés par AWS, non modifiables.
Sert à	Donner des droits à votre code ou à votre ressource.	Permettre au service de faire son propre travail dans votre compte.
Nom	Libre : <code>shop-web-role</code> .	<code>AWSServiceRoleFor...</code>
Suppression	Libre.	Seulement quand le service ne l'utilise plus.

Service	Principal dans la trust policy
EC2	<code>ec2.amazonaws.com</code>
Lambda	<code>lambda.amazonaws.com</code>
Tâches ECS	<code>ecs-tasks.amazonaws.com</code>
Pods EKS (Pod Identity)	<code> pods.eks.amazonaws.com</code>
CloudFormation	<code>cloudformation.amazonaws.com</code>
EventBridge	<code>events.amazonaws.com</code>

Exemples de rôles liés. `AWSServiceRoleForAutoScaling` lance et arrête des instances ; `AWSServiceRoleForRDS` gère les interfaces réseau des bases. On les voit dans la liste des rôles sans les avoir créés : c'est normal, et il ne faut pas y toucher.

Définition : rôle lié à un service (service-linked role)
Rôle prédéfini, rattaché à un service AWS précis, que ce service crée et utilise pour gérer des ressources en votre nom.

Définition : instance profile

Conteneur qui porte un rôle IAM et que l'on attache à une instance EC2. C'est par lui que l'instance reçoit les identifiants du rôle.

Définition : service de métadonnées (IMDS)

Service local à chaque instance, joignable à l'adresse 169.254.169.254, qui lui fournit des informations sur elle-même, dont les identifiants temporaires de son rôle.

À savoir

- Une instance porte un seul instance profile, donc un seul rôle.
- La console crée l'instance profile en même temps que le rôle, sous le même nom ; en CLI, ce sont deux objets à créer.
- On peut attacher, remplacer ou retirer le rôle d'une instance en marche.

IMDSv2

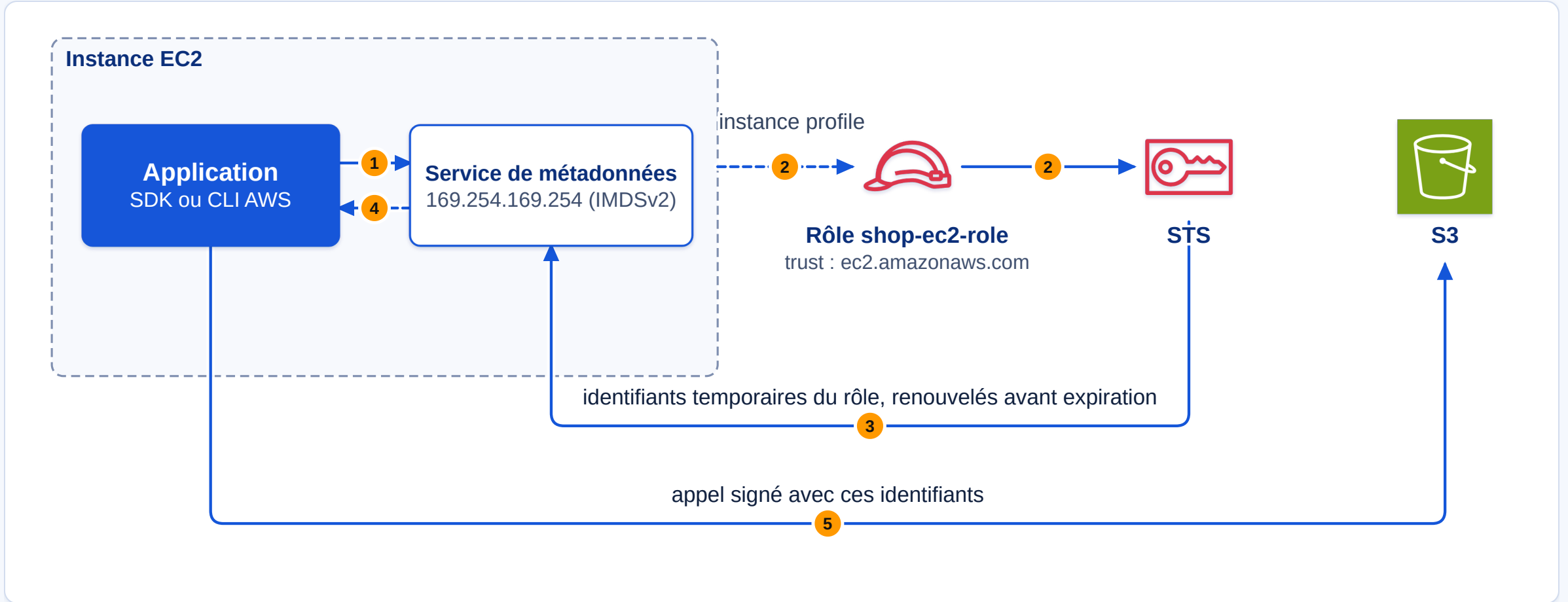
- La version 2 exige de demander d'abord un jeton de session, puis de le présenter : cela bloque une famille d'attaques où un site web est amené à lire les métadonnées à la place de l'application.
- C'est le réglage par défaut des images récentes, et celui à exiger partout.

\$ Depuis l'instance : voir le rôle, sans jamais afficher les secrets

```
TOKEN=$(curl -s -X PUT http://169.254.169.254/latest/api/token \
-H "X-aws-ec2-metadata-token-ttl-seconds: 300")
curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
http://169.254.169.254/latest/meta-data/iam/security-credentials/

# Le plus simple : demander à la CLI qui elle est
aws sts get-caller-identity
```

Schéma : comment une instance EC2 obtient ses identifiants



1. Le SDK ou la CLI, ne trouvant ni variable ni fichier, interroge le service de métadonnées.

2. EC2 a endossé pour l'instance le rôle porté par son instance profile.

3. STS délivre des identifiants temporaires, qu'EC2 met à disposition et renouvelle avant leur expiration.

4. Le service de métadonnées les remet à l'application.

5. L'application signe ses appels avec eux : elle a les droits du rôle, ni plus ni moins.

EC2 : créer et attacher un rôle en ligne de commande

\$ Prérequis : droits iam:CreateRole, iam:PassRole, et une instance existante

1. Trust policy : le service EC2 peut endosser le rôle

```
cat > ec2-trust.json <<'EOF'
{"Version":"2012-10-17","Statement":[{"Effect":"Allow",
  "Principal":{"Service":"ec2.amazonaws.com"},
  "Action":"sts:AssumeRole"}]}
EOF
```

2. Le rôle, puis ses droits

```
aws iam create-role --role-name user7-role-ec2-s3 \
  --assume-role-policy-document file://ec2-trust.json
aws iam attach-role-policy --role-name user7-role-ec2-s3 \
  --policy-arn arn:aws:iam::aws:policy/AmazonS3ReadOnlyAccess
```

3. L'instance profile, qui contient le rôle

```
aws iam create-instance-profile \
  --instance-profile-name user7-role-ec2-s3
aws iam add-role-to-instance-profile \
  --instance-profile-name user7-role-ec2-s3 \
  --role-name user7-role-ec2-s3
```

4. L'attacher à l'instance

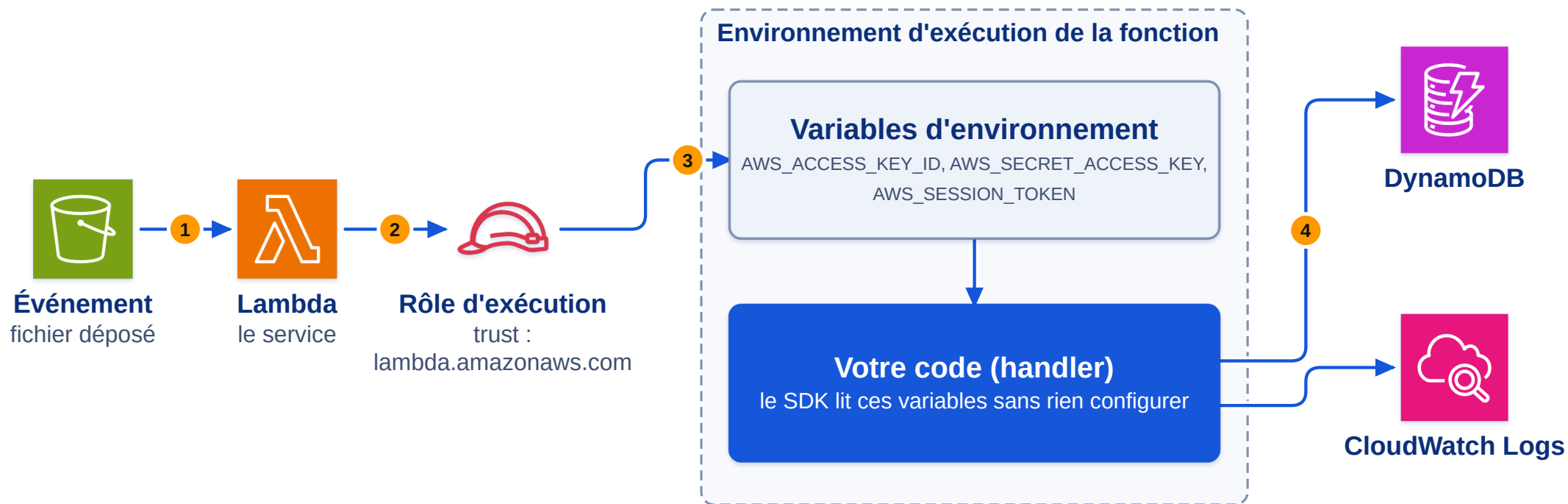
```
aws ec2 associate-iam-instance-profile \
  --instance-id i-0abc123def4567890 \
  --iam-instance-profile Name=user7-role-ec2-s3
```

#	Ce que fait l'étape
1	Écrit la trust policy : qui peut endosser.
2	Crée le rôle et lui attache une politique de permissions : ce qu'il peut faire.
3	Crée l'enveloppe qu'EC2 sait manipuler, et y place le rôle.
4	Relie l'enveloppe à l'instance. Cette étape exige <code>iam:PassRole</code> .

Vérification. Depuis l'instance, `aws sts get-caller-identity` renvoie `assumed-role/user7-role-ec2-s3/i-0abc...` : le nom de session est l'identifiant de l'instance.

L'identifiant d'instance est un exemple : il se remplace par celui de votre propre instance. Dans la console, tout se fait par **Actions, Security, Modify IAM role**.

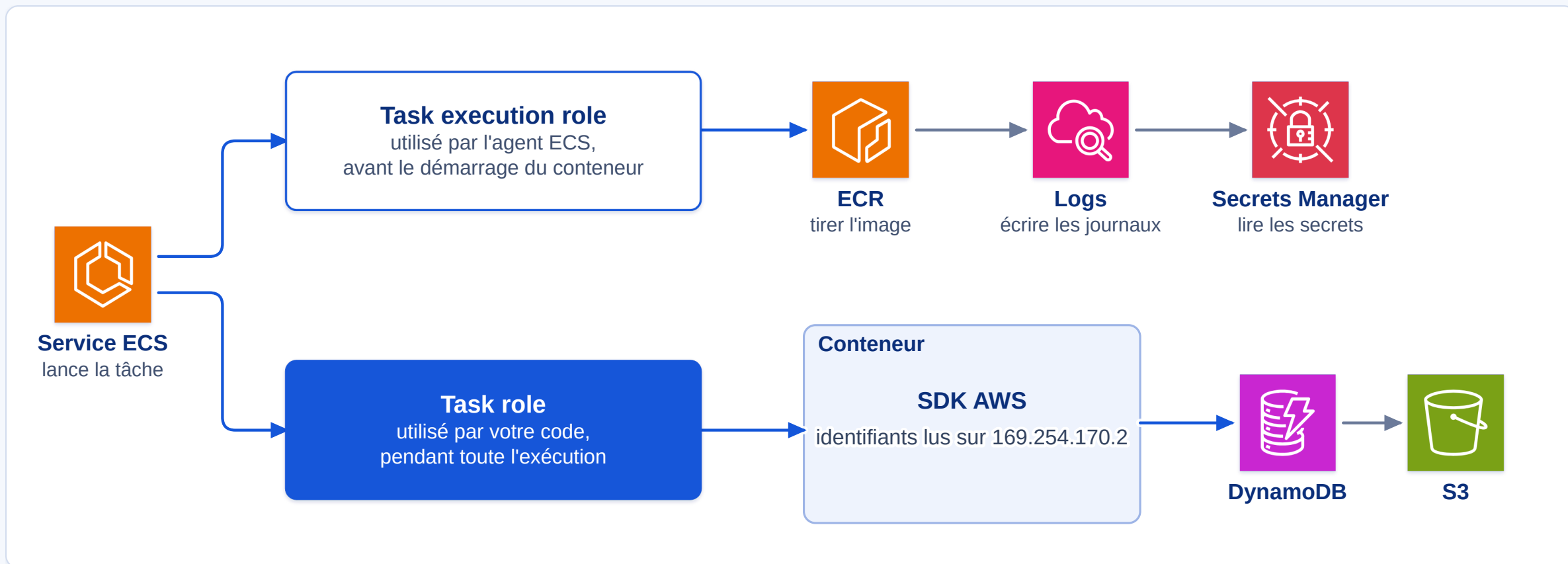
Schéma : le rôle d'exécution d'une fonction Lambda

**Définition : rôle d'exécution**

Rôle IAM que le service Lambda endosse pour chaque fonction, et dont le code de la fonction reçoit les identifiants temporaires.

1. Un événement arrive : Lambda doit exécuter la fonction.
2. Lambda endosse le rôle d'exécution désigné dans la configuration de la fonction.
3. Les identifiants temporaires sont placés dans les variables d'environnement.
4. Le code appelle DynamoDB, écrit ses journaux : tout cela avec les droits du rôle.

Schéma : les deux rôles d'une tâche ECS



Task execution role

Sert à **préparer** la tâche : télécharger l'image, créer les flux de journaux, lire les secrets à injecter. Votre code ne l'utilise jamais. Politique habituelle :

`AmazonECSTaskExecutionRolePolicy`.

Task role

Sert à votre **application**, pendant qu'elle tourne. Le SDK récupère ses identifiants sur une adresse locale au conteneur. Un rôle par service applicatif, au moindre privilège.

Rôles de service

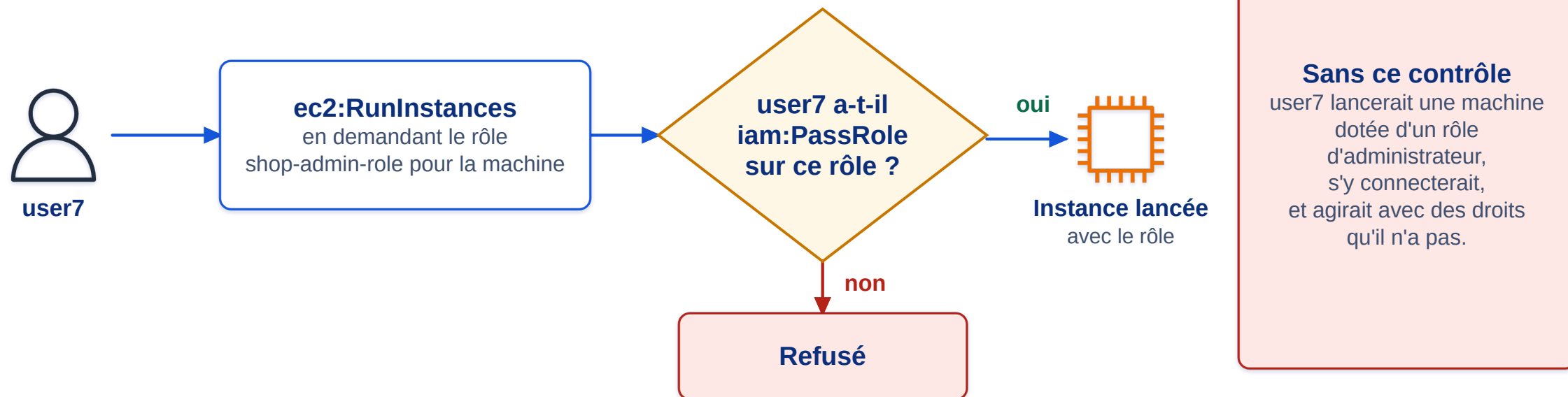
EC2, Lambda, ECS : le même principe, trois mécanismes

	EC2	Lambda	ECS
Nom du rôle	Rôle d'instance	Rôle d'exécution	Task role (et execution role)
Principal de confiance	<code>ec2.amazonaws.com</code>	<code>lambda.amazonaws.com</code>	<code>ecs-tasks.amazonaws.com</code>
Rattachement	Instance profile sur l'instance	Champ <code>Role</code> de la fonction	<code>taskRoleArn</code> de la task definition
Où le code trouve les identifiants	Service de métadonnées, 169.254.169.254	Variables d'environnement	Point d'accès local, 169.254.170.2
Granularité	Un rôle par instance	Un rôle par fonction	Un rôle par tâche
Nom de session	Identifiant de l'instance	Nom de la fonction	Identifiant de la tâche

Ce qui ne change pas. Une trust policy qui désigne le service, des politiques de permissions au moindre privilège, des identifiants temporaires renouvelés automatiquement, et un code sans aucun secret.

Conséquence de la granularité. Sur EC2, tous les programmes de l'instance partagent le rôle. Avec ECS et Lambda, chaque composant a le sien : le cloisonnement est plus fin.

Schéma : iam:PassRole, le droit de confier un rôle

**Définition : iam:PassRole**

Autorisation requise pour désigner un rôle à un service AWS lors de la création ou de la configuration d'une ressource. Ce n'est pas un appel d'API, mais un contrôle fait par le service.

```
{ } Autoriser à ne confier que ses propres rôles, et seulement à EC2

{ "Effect": "Allow",
  "Action": "iam:PassRole",
  "Resource": "arn:aws:iam::123456789012:role/user7-*",
  "Condition": { "StringEquals": {
    "iam:PassedToService": "ec2.amazonaws.com" } } }
```

Schéma : la chaîne de recherche des identifiants

La CLI et les SDK essaient chaque source dans l'ordre et s'arrêtent à la première qui fournit des identifiants.



sur un poste de travail

sur une ressource AWS : c'est le rôle de service qui répond

Le même code fonctionne donc sur le poste du développeur et en production, sans modification.

Définition : chaîne de fournisseurs d'identifiants

Suite ordonnée de sources que la CLI et les SDK consultent pour trouver des identifiants, en s'arrêtant à la première qui en fournit.

Piège. Une variable `AWS_ACCESS_KEY_ID` oubliée dans l'environnement d'une instance passe avant son rôle. Le code agit alors avec une autre identité que celle prévue : `aws sts get-caller-identity` le révèle aussitôt.

Symptôme	Piste
« Unable to locate credentials »	Aucun rôle attaché, ou le service de métadonnées est injoignable.
<code>AccessDenied</code> depuis l'application	Lire l'ARN du message : est-ce le bon rôle ? Puis ajouter l'action et la ressource citées.
« is not authorized to perform: iam:PassRole »	Celui qui crée la ressource ne peut pas lui confier ce rôle.
Le service refuse d'endosser le rôle	La trust policy ne cite pas le bon service.
Rôle créé à l'instant, refusé	Propagation d'IAM : attendre quelques secondes.
Droits modifiés, sans effet apparent	Vérifier que l'on modifie le rôle réellement utilisé.

Bonnes pratiques

- Un rôle par application ou par service, jamais un rôle commun.
- Des politiques qui nomment les ressources : ce bucket, cette table.
- Aucune clé d'accès dans le code, les images, les variables d'environnement ou les dépôts.
- `iam:PassRole` limité à des rôles nommés et à un service donné.
- IMDSv2 exigé sur toutes les instances.
- Nom du rôle selon la convention : projet, environnement, usage.

Premier geste de diagnostic. `aws sts get-caller-identity`, exécuté là où tourne le code : il dit sans ambiguïté sous quelle identité les appels partent.

Ce qu'il faut retenir

- La CLI a besoin de trois choses : une identité, une région, un format de sortie.
- `aws login` donne des identifiants temporaires à partir de la connexion console, sans clé permanente.
- Un profil regroupe ces réglages ; l'option l'emporte sur la variable, qui l'emporte sur le fichier.
- `--filters` réduit côté serveur, `--query` extrait côté client, `--output text` alimente les scripts.
- Endosser un rôle, c'est demander à STS des identifiants temporaires ; il faut l'accord de la trust policy.
- EC2, Lambda et ECS reçoivent leurs droits par un rôle de service : aucun secret dans le code.
- `aws sts get-caller-identity` répond toujours à la question « qui suis-je ? ».

1. Quelles sont les trois informations dont la CLI a besoin ?

2. Entre `--region`, `AWS_REGION` et le profil, qui l'emporte ?

3. Quelle différence entre `--filters` et `--query` ?

4. Que fait `Reservations[].Instances[]` ?

5. Que contient un jeu d'identifiants temporaires ?

6. Quel document dit qui peut endosser un rôle ?

7. Où le code d'une instance EC2 trouve-t-il ses identifiants ?

Cliquer sur une question pour afficher la réponse.

Les bonnes pratiques IAM en une page

#	Pratique	Où elle a été vue
1	Verrouiller l'utilisateur racine : second facteur, aucune clé, usage exceptionnel.	Premiers pas dans la console
2	Second facteur pour toutes les personnes.	Premiers pas dans la console
3	Identifiants temporaires partout : fédération pour les personnes, rôles pour le code.	Tout ce support
4	Aucune clé d'accès à longue durée, sauf impossibilité démontrée.	IAM, les fondamentaux
5	Droits accordés aux groupes et aux rôles, pas aux individus.	IAM, les fondamentaux
6	Moindre privilège : actions, ressources et conditions resserrées par itérations.	IAM, les fondamentaux
7	Politiques gérées par AWS pour démarrer, gérées par le client pour préciser.	IAM, les fondamentaux
8	Conditions et garde-fous pour renforcer : second facteur, région, permissions boundary, SCP.	IAM, les fondamentaux
9	Vérifier et réduire : Access Analyzer, dernier accès, rapport d'identifiants.	IAM, les fondamentaux
10	Tout journaliser avec CloudTrail, et regarder ce qui est journalisé.	Chapitre sur la supervision

Fil conducteur. Réduire ce que chaque identité peut faire, réduire la durée pendant laquelle elle peut le faire, et garder la trace de ce qu'elle a fait.

Terme	Définition
Authentification	Vérification de l'identité de l'auteur d'une requête.
Autorisation	Décision d'accepter ou de refuser une action, d'après les politiques.
Principal	Entité authentifiable qui envoie une requête : utilisateur, session de rôle, service.
Utilisateur IAM	Identité dotée d'identifiants permanents, pour une personne ou une application.
Groupe IAM	Ensemble d'utilisateurs auquel on attache des politiques.
Rôle IAM	Identité sans identifiants permanents, endossée temporairement.
Politique	Document JSON d'autorisations et de refus.
Déclaration	Règle d'une politique : effet, action, ressource, condition.
Refus par défaut	Tout ce qui n'est pas autorisé explicitement est refusé.
Refus explicite	Déclaration Deny, qui l'emporte sur toute autorisation.
Moindre privilège	N'accorder que les droits strictement nécessaires.
Politique gérée	Politique autonome, attachable à plusieurs identités ; écrite par AWS ou par le client.
Politique en ligne	Politique incluse dans une seule identité.
Permissions boundary	Plafond de droits d'un utilisateur ou d'un rôle.
ABAC	Contrôle d'accès fondé sur la correspondance d'étiquettes.

Terme	Définition
AWS CLI	Outil en ligne de commande qui appelle les API d'AWS.
Profil	Ensemble nommé de réglages : identité, région, format de sortie.
Clé d'accès	Identifiant et secret permanents d'un utilisateur IAM.
Identifiants temporaires	Clé, secret et jeton de session à durée limitée, délivrés par STS.
STS	Service qui délivre les identifiants temporaires.
Trust policy	Politique d'un rôle qui dit quels principaux peuvent l'endosser.
AssumeRole	Opération de STS par laquelle on endosse un rôle.
Session	Période de validité d'identifiants temporaires, identifiée par un nom.
Fédération	Authentification par un fournisseur d'identité externe, suivie de la prise d'un rôle.
JMESPath	Langage de requête sur du JSON, utilisé par <code>--query</code> .
Rôle de service	Rôle endossé par un service AWS pour agir en votre nom.
Rôle lié à un service	Rôle prédéfini, créé et utilisé par un service pour son propre fonctionnement.
Instance profile	Conteneur qui rattache un rôle à une instance EC2.
IMDS	Service de métadonnées d'une instance, qui lui remet les identifiants de son rôle.
iam:PassRole	Droit de confier un rôle à un service.

Sujet	Documentation officielle
IAM	IAM User Guide
Éléments d'une politique	IAM JSON policy elements reference
Logique d'évaluation	Policy evaluation logic
Actions, ressources, clés	Service Authorization Reference
Bonnes pratiques	Security best practices in IAM
AWS CLI	AWS CLI User Guide
JMESPath	jmespath.org/tutorial.html
STS	STS API Reference

Icônes. Les icônes des schémas proviennent du jeu officiel « AWS Architecture Icons », publié par AWS pour la réalisation de schémas d'architecture.

Suite du parcours : le réseau

- VPC et sous-réseaux.
- Routage et accès à Internet.
- Filtrage et accès privés.
- Méthode de dépannage réseau.

Ce qui sera réutilisé ensuite

- Le profil CLI qui endosse `RoleApprenants`.
- `--query` et `--output text` pour récupérer des identifiants de VPC et de sous-réseaux.
- La lecture d'un `AccessDenied`.
- Les rôles de service, pour EC2, puis ECS, puis Lambda.

Fin de cette partie. Les deux premiers supports couvrent les fondations et la gestion des accès.